

Web-first big data visualization & analytics

CS524: Big Data Visualization & Analytics
Fabio Miranda



urbantk.org



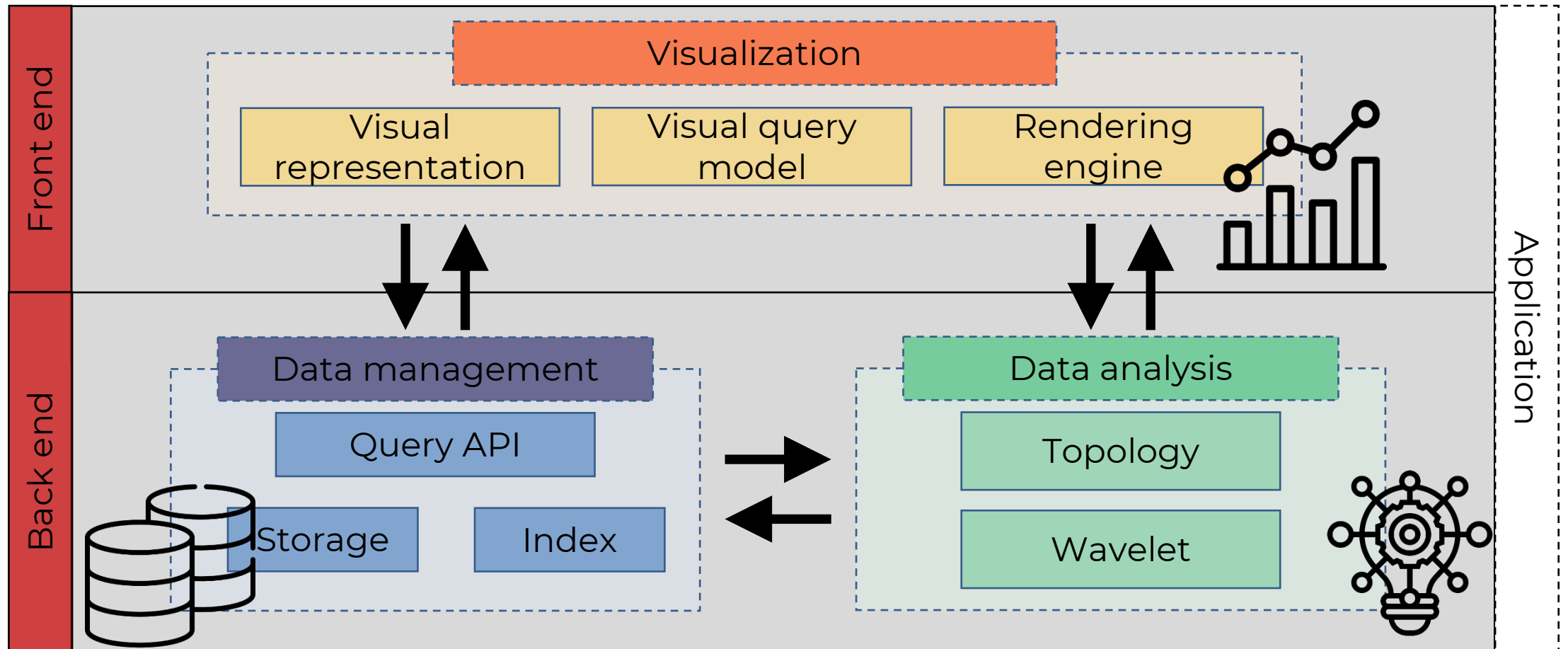
evl

electronic
visualization
laboratory



UNIVERSITY OF
ILLINOIS CHICAGO

Big data visualization system



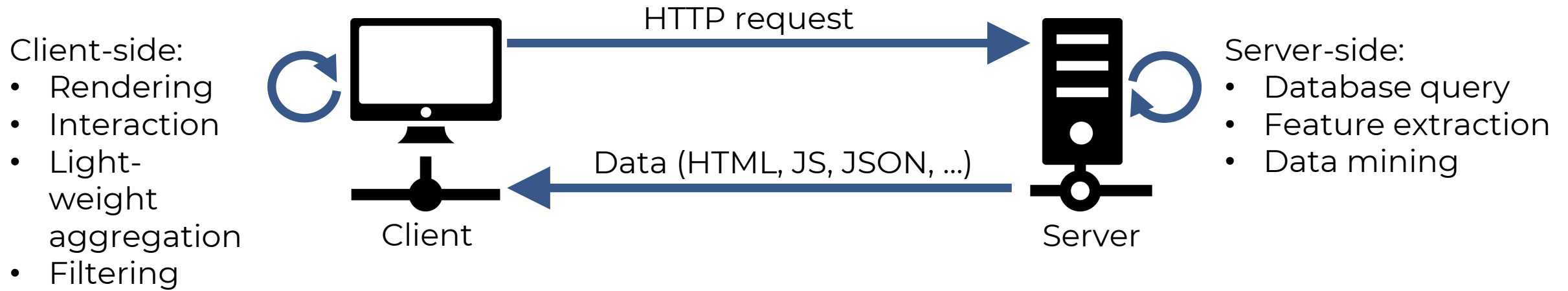
Web-first big data visualization system

- Why separate front-end and back-end development?
 - Separation of concerns between presentation layer (front end) and data layer (back end).
 - Easily mapped to a client-server model.
 - Client: front end
 - Server: back end
 - Easy deployment.

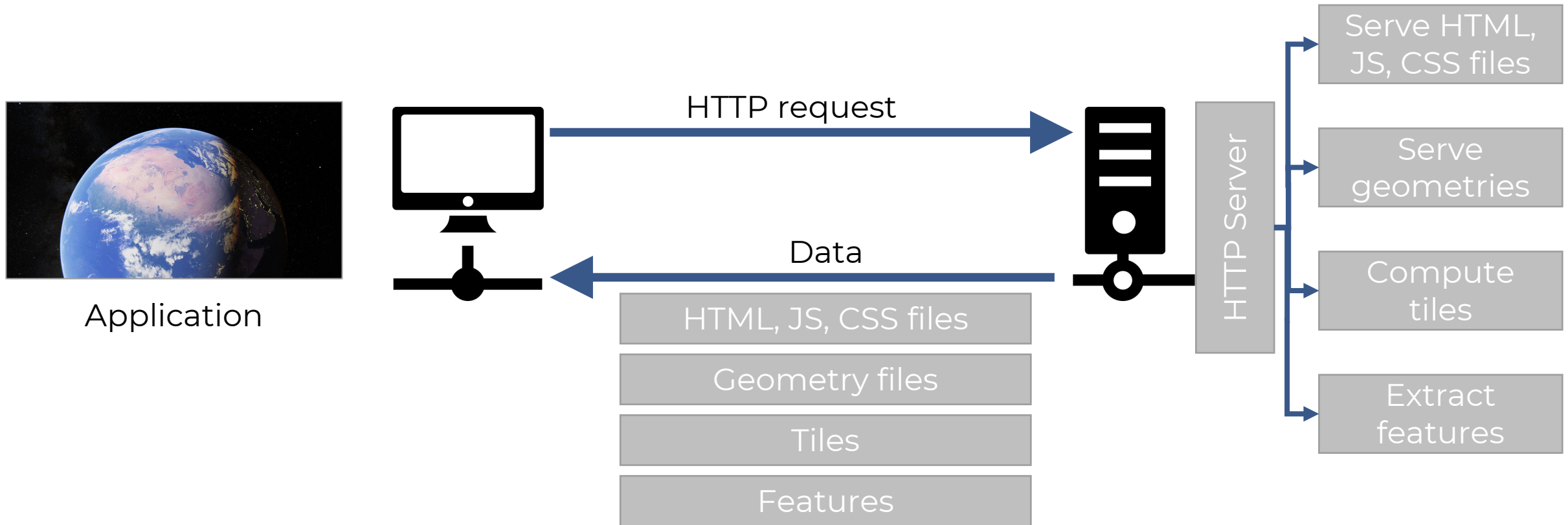
Building blocks

- Front-end:
 - Web technologies
 - Web environment
 - JavaScript
 - D3
 - TypeScript
 - Angular
- Front-end and back-end communication:
 - Flask (Python)
 - Mongoose (C / C++)
- Back-end building blocks:
 - Boost
 - Qt
 - CUDA

Client and server



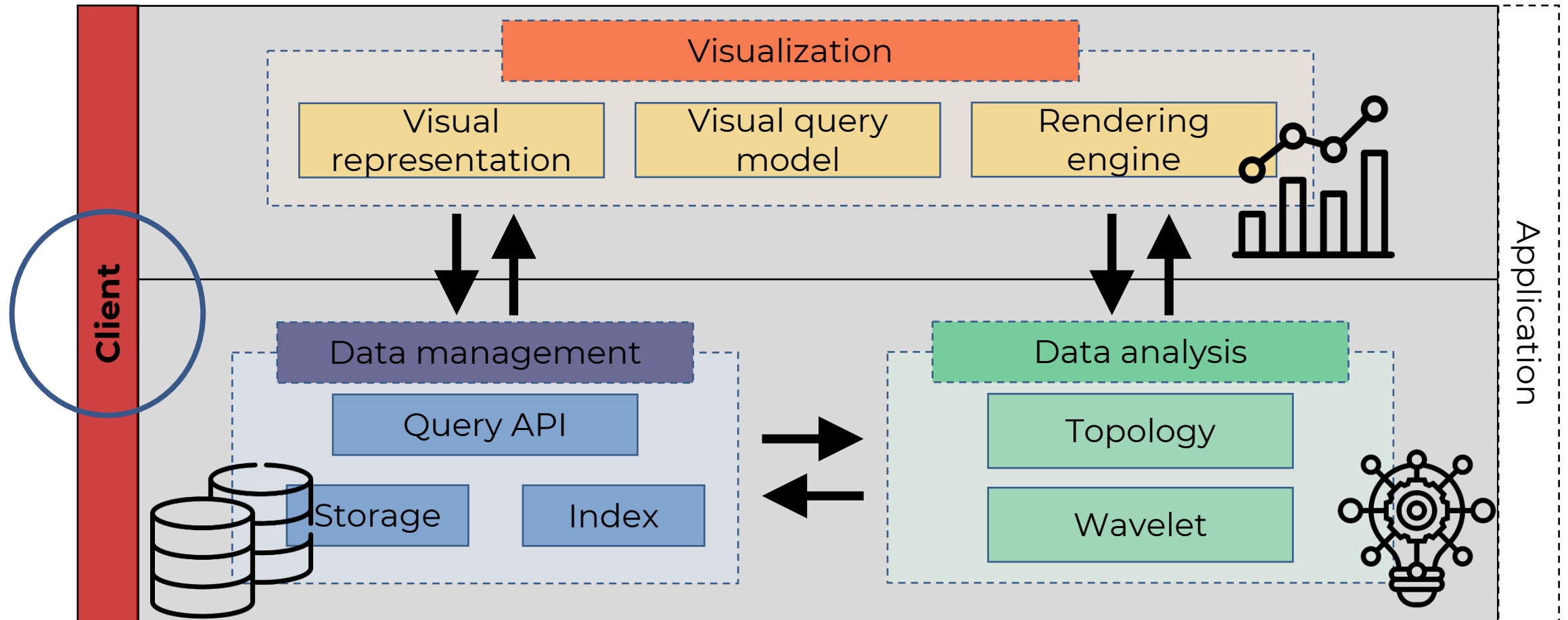
Client and server



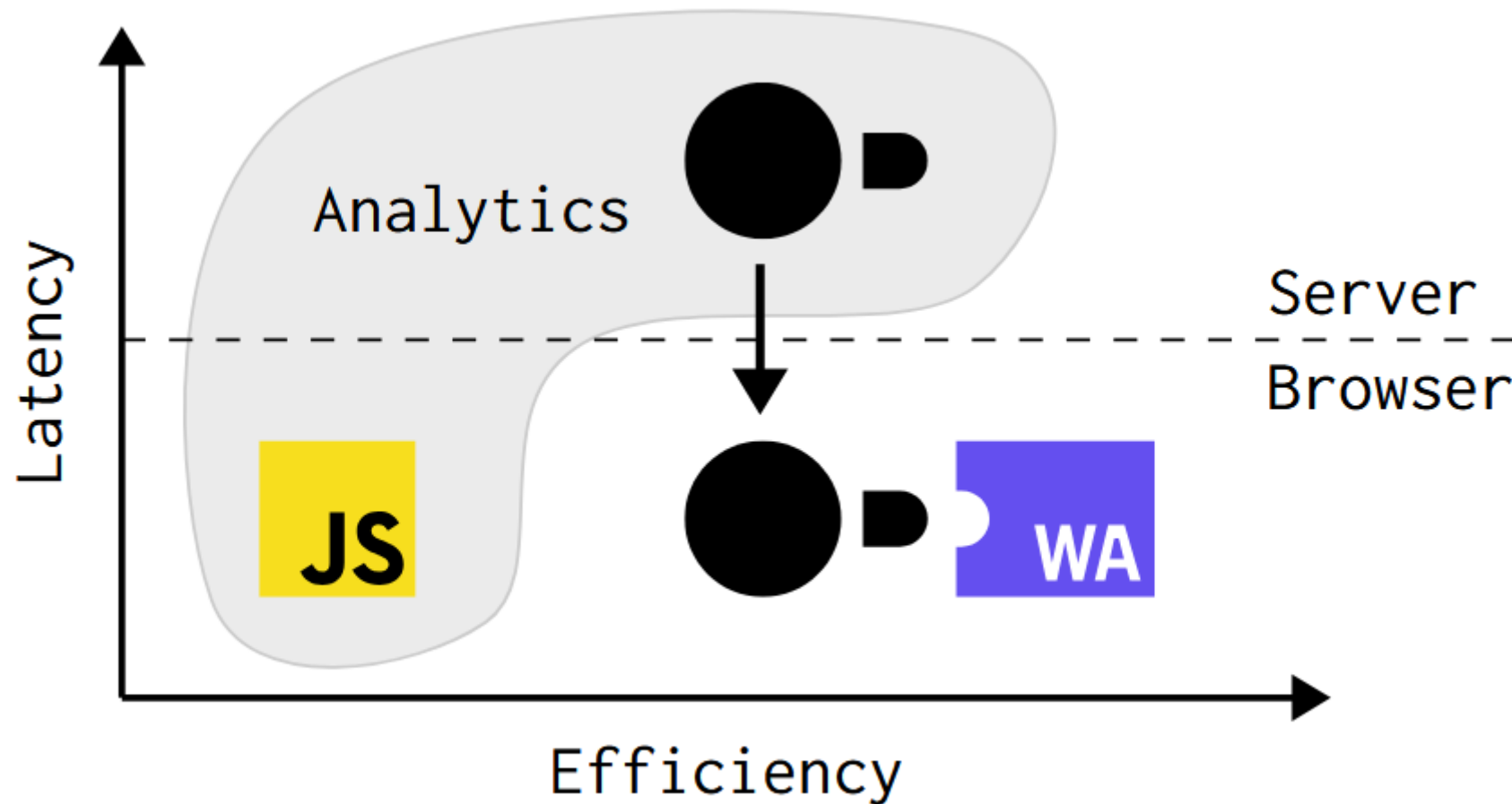
Do we really need client and server separation?
How to support interactivity in a web environment?



Big data visualization system

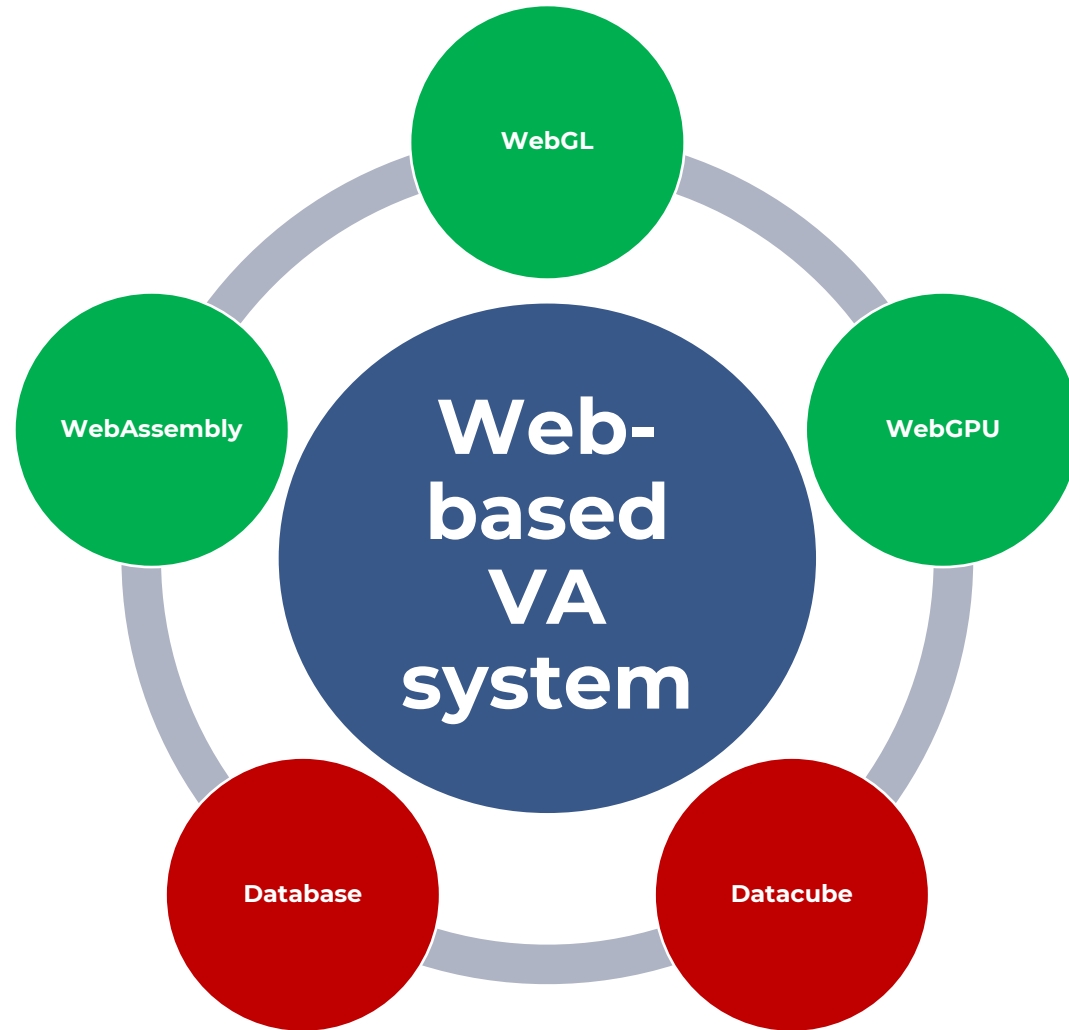


How to support interactivity in a web environment?



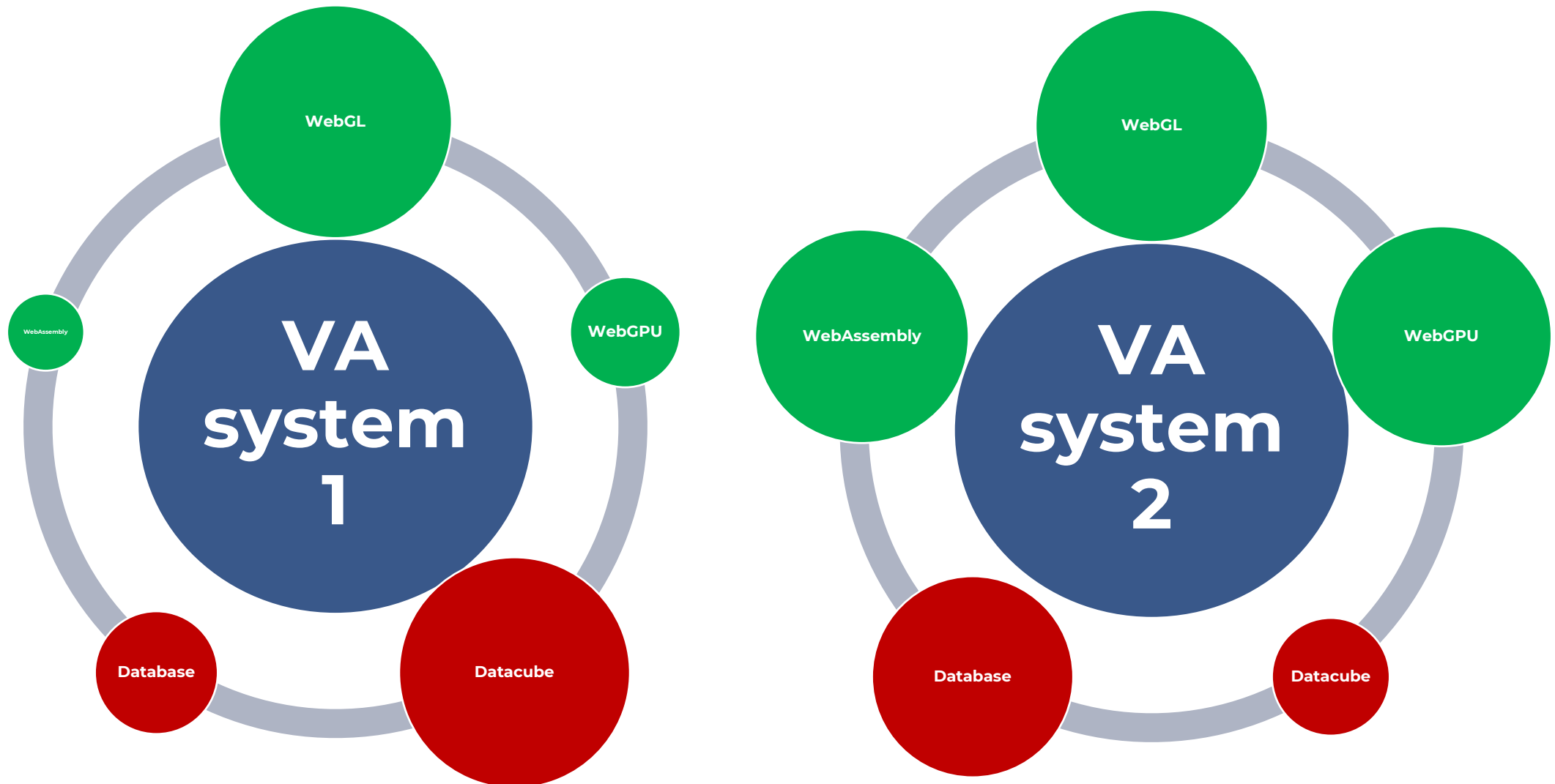
Kohn et al., 2022

How to support interactivity in a web environment?



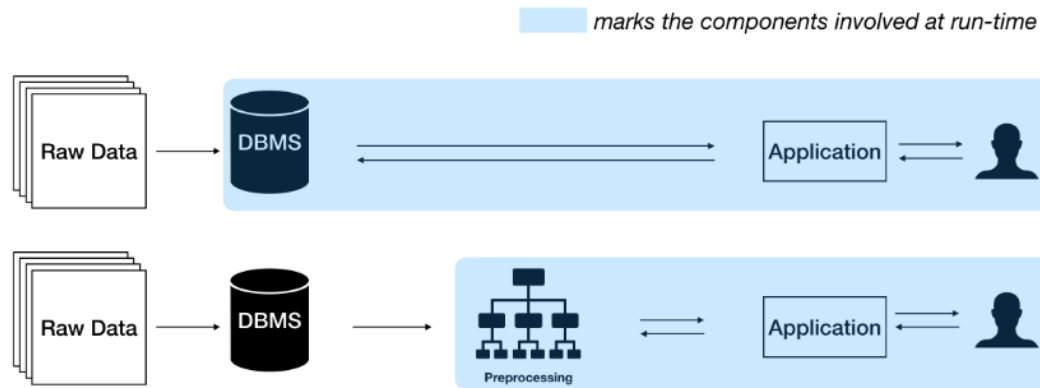
- **Backend computation:**
 - **Datacube:** Precompute a multi-dimensional cube (space / time / attributes) so interactions become tiny lookups.
 - **Database:** Run queries and heavy analytics near the data (DB / cluster) and send back small, interaction-ready results.
- **Frontend computation:**
 - **WebGL:** Use the GPU for fast rendering in the browser to keep millions of marks interactive.
 - **WebGPU:** Lower-level GPU API for compute-friendly rendering and data processing.
 - **WebAssembly:** Compile C/C++/Rust to run near-native compute in the browser for tight loops

How to support interactivity in a web environment?



Backend computation

- Keep the browser *thin* by pushing filtering, joints, and aggregations down to the compute server.
- Return summaries (aggregates, bins, tiles) instead of raw records.
- Two broad strategies:
 1. Datacube-based pre-aggregations.
 2. DB-based query pushdown.



NeuralCubes, Wang et al., 2021

Nanocubes, Lins et al., 2013

2456 IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 19, NO. 12, DECEMBER 2013

Nanocubes for Real-Time Exploration of Spatiotemporal Datasets

Lauro Lins, James T. Klosowski, and Carlos Scheidegger

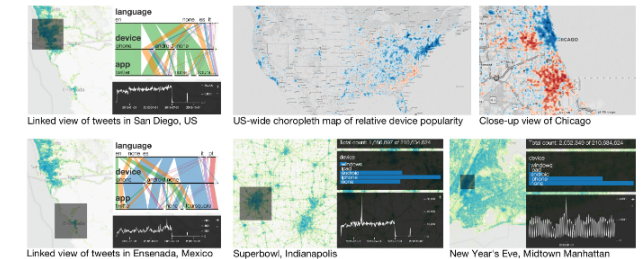


Fig. 1. Example visualizations of 210 million public geolocated Twitter posts over the course of a year. The data structure we propose enables real-time (these images above were rendered faster than the typical screen refresh rate) visual exploration of large, spatiotemporal, multidimensional datasets. The visual encodings built using *nanocubes* are within a controllable difference to ones rendered by a traditional linear scan over the dataset. They naturally support linked navigation and brushing, and include choropleth maps, time series over arbitrary regions and scales of space and time, parallel sets, histograms, and binned scatterplots. The color scale of the choropleth map is a diverging scale in which blue corresponds to iPhones being relatively more popular, and red corresponds to higher relative popularity of Android devices.

Abstract—Consider real-time exploration of large multidimensional spatiotemporal datasets with billions of entries, each defined by a location, a time, and other attributes. Are certain attributes correlated spatially or temporally? Are there trends or outliers in the data? Answering these questions requires aggregation over arbitrary regions of the domain and attributes of the data. Many relational databases implement the well-known *data cube* aggregation operation, which in a sense precomputes every possible aggregate query over the database. Data cubes are sometimes assumed to take a prohibitively large amount of space, and to consequently require disk storage. In contrast, we show how to construct a data cube that fits in a modern laptop's main memory, even for billions of entries; we call this data structure a *nanocube*. We present algorithms to compute and query a *nanocube*, and show how it can be used to generate well-known visual encodings such as heatmaps, histograms, and parallel coordinate plots. When compared to exact visualizations created by scanning an entire dataset, *nanocube* plots have bounded screen error across a variety of scales, thanks to a hierarchical structure in space and time. We demonstrate the effectiveness of our technique on a variety of real-world datasets, and present memory, timing, and network bandwidth measurements. We find that the timings for the queries in our examples are dominated by network and user-interaction latencies.

Index Terms—Data cube, Data structures, Interactive exploration

1 INTRODUCTION

As datasets get larger, exploratory data visualization becomes more difficult. Consider a dataset with a billion entries. We can compute a small summary of the dataset and visualize the summary instead of the dataset, but as Anscombe's famous quartet shows [3], summaries themselves cannot ascertain their own validity. Summaries *might* help, but in order to understand if that is the case, we will inevitably find

ourselves having to visualize one billion residuals. As far as scale goes, we are back to square one. In other words, data summarization alone will never solve the problem of scale in exploratory visualization. As visualization practitioners, what then can we do? Even drawing the simplest scatterplot is not straightforward. If we decide to produce the visualization by scanning the rows of a table, we will either need non-trivial parallel rendering algorithms or significant time to produce a drawing. Neither of these solutions is attractive or scales well with dataset size.

- Lauro Lins is with AT&T Research. E-mail: lins@research.att.com.
- Jim Klosowski is with AT&T Research. E-mail: jklosow@research.att.com.
- Carlos Scheidegger is with AT&T Research. E-mail: cscheide@research.att.com.

Manuscript received 31 March 2013; accepted 1 August 2013; posted online 13 October 2013; mailed on 4 October 2013.
For information on obtaining reprints of this article, please send e-mail to: tvcg@computer.org.

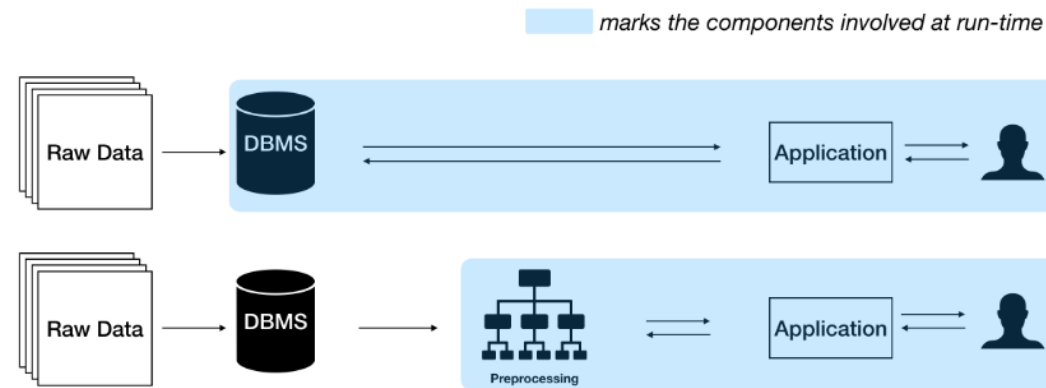
Data cubes are structures that perform aggregations across every possible set of dimensions of a table in a database, to support quick exploration [15, 31]. Many visualization systems are built on top of data cubes, concretely or conceptually. Still, only recently have researchers started to examine data cube creation algorithms in the context of information visualization [33, 18, 21].

Data cubes are often problematic in that they can take prohibitively large amounts of memory as the number of dimensions increases. In

Authorized licensed use limited to: University of Illinois at Chicago. Downloaded on January 20, 2025 at 00:53:33 UTC from IEEE Xplore. Restrictions apply.

Backend computation

- Keep the browser *thin* by pushing filtering, joints, and aggregations down to the compute server.
- Return summaries (aggregates, bins, tiles) instead of raw records.
- Two broad strategies:
 1. Datacube-based pre-aggregations.
 2. DB-based query pushdown.

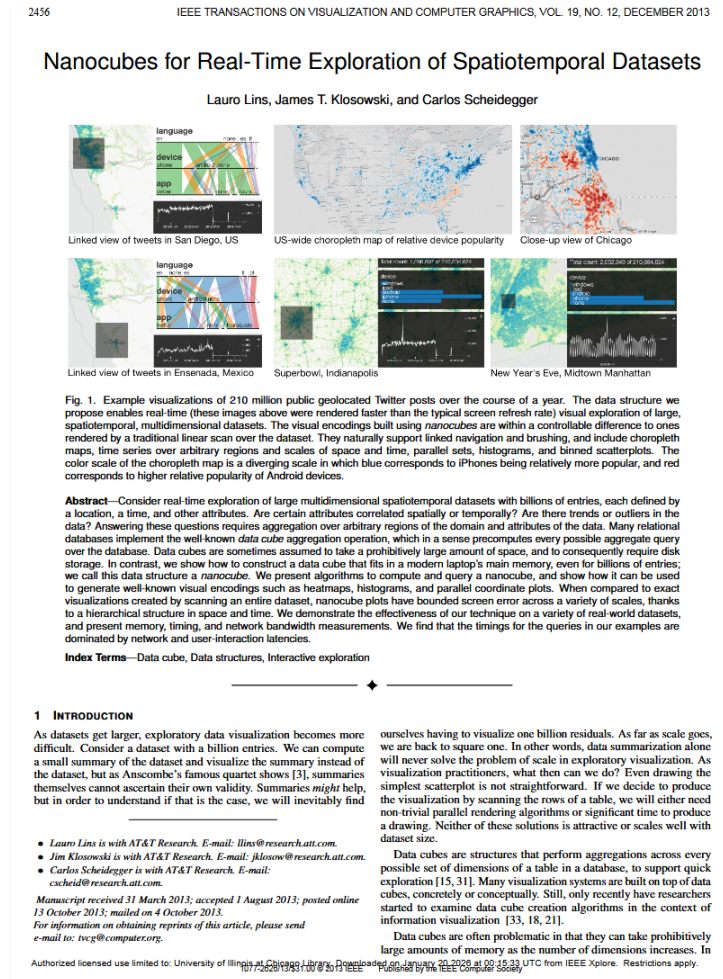


NeuralCubes, Wang et al., 2021

Backend computation: Datacubes

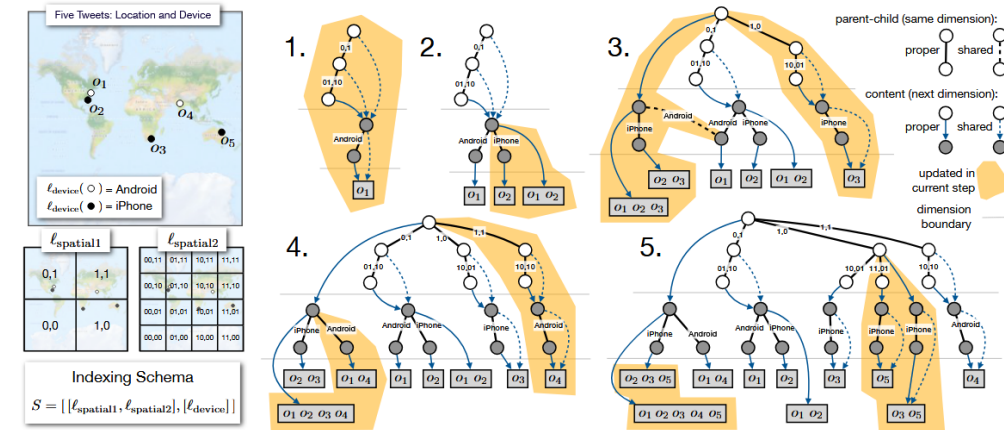
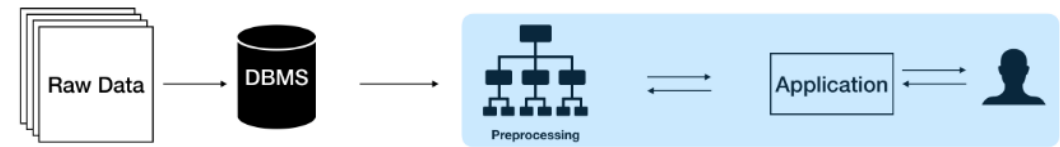
- Datacube-based approach:
 - Precompute a multi-dimensional cube (space / time / attributes) so interactions become tiny lookups.
 - Ultra-low latency for common operations: counts, histograms, heatmaps, time series, rollups across levels.
 - Multi-resolution by design: serve aggregates at different zoom levels without rerunning expensive queries.

Nanocubes, Lins et al., 2013



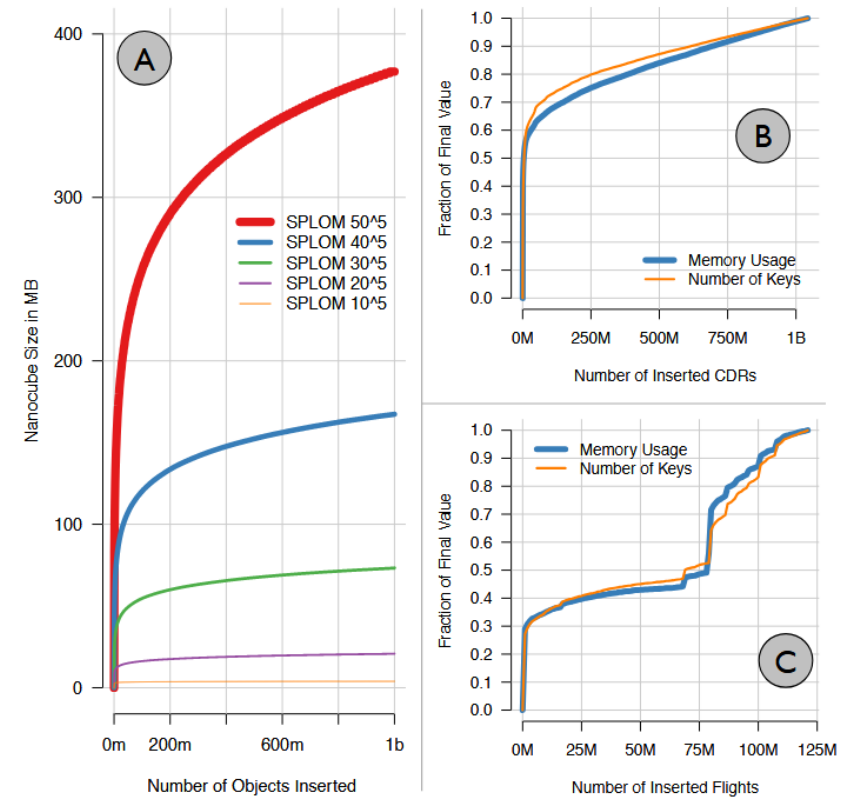
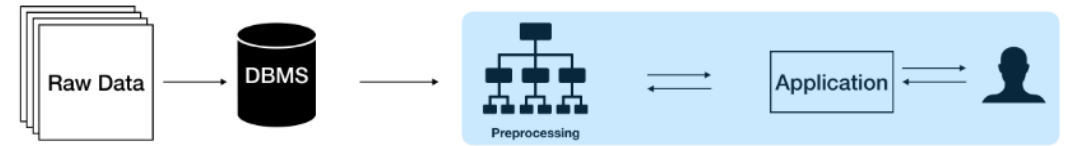
Backend computation: Datacubes

- Datacube-based approach:
 - Precompute a multi-dimensional cube (space / time / attributes) so interactions become tiny lookups.
 - Ultra-low latency for common operations: counts, histograms, heatmaps, time series, rollups across levels.
 - Multi-resolution by design: serve aggregates at different zoom levels without rerunning expensive queries.



Backend computation: Datacubes

- Datacube-based approach:
 - **Tradeoffs:** Less flexibility for arbitrary queries / joins ; cube design must anticipate dimensions and binning.
 - **Cost:** memory footprint + cube rebuild / update when data changes.



Backend computation: Database

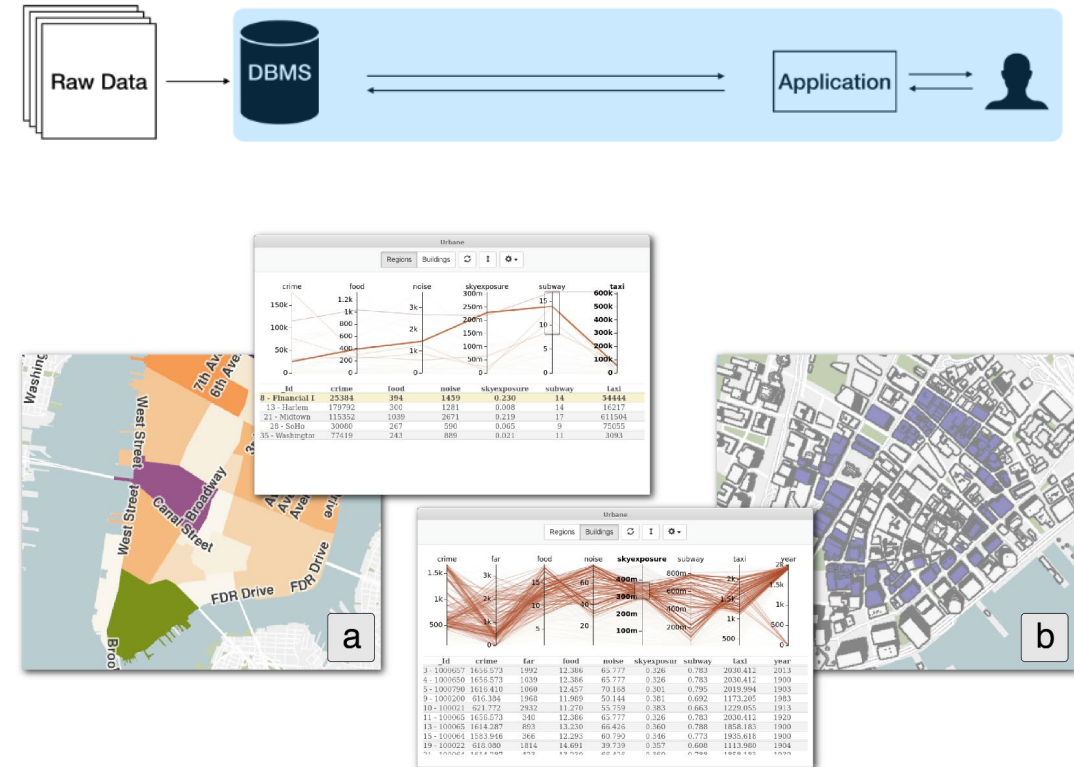
- DB-based approach:
 - Run interactive queries directly in the DB/engine: GROUP BY, filters, spatial predicates, windowed time ranges.
 - Flexibility: supports ad hoc slicing, joins with external tables, and evolving analysis questions.
 - Performance comes from engineering: indexes (spatial/temporal), column stores, partitioning, materialized views.
 - Latency control techniques: query/result caching keyed by viewport+filters; pre-aggregations for “hot” interactions.



Doraiswamy et al., 2018

Backend computation: Database

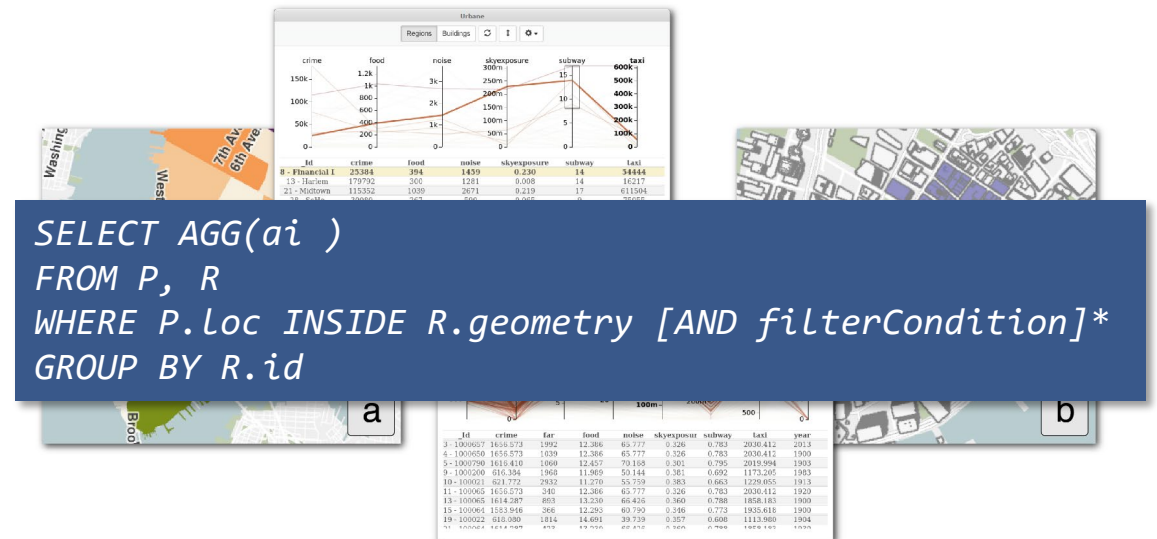
- Raster Join:
 - Reframes spatial aggregation (points-in-polygon + group-by) as a drawing points and polygons on the same GPU canvas, then combining join + aggregation.
 - Rasterize points into an FBO, rasterize polygons / pixels then accumulate.



Doraiswamy et al., 2018

Backend computation: Database

- Raster Join:
 - Reframes spatial aggregation (points-in-polygon + group-by) as a drawing points and polygons on the same GPU canvas, then combining join + aggregation.
 - Rasterize points into an FBO, rasterize polygons / pixels then accumulate.



Doraiswamy et al., 2018

Backend computation: Database

- Raster Join:
 - Reframes spatial aggregation (points-in-polygon + group-by) as a drawing points and polygons on the same GPU canvas, then combining join + aggregation.
 - Rasterize points into an FBO, rasterize polygons / pixels then accumulate.

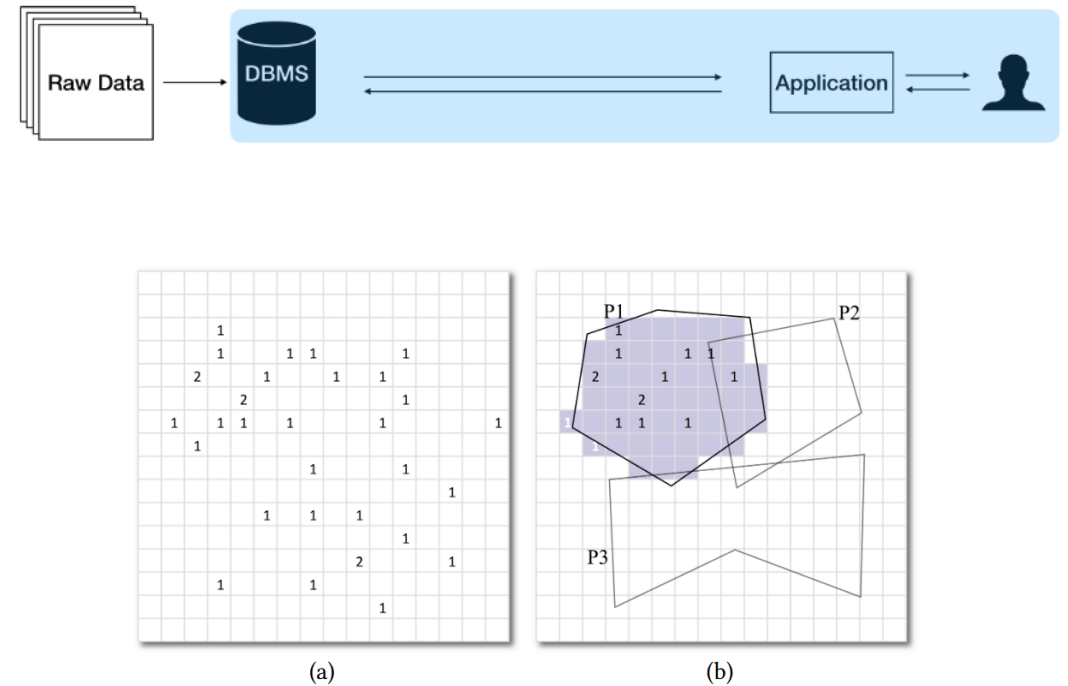
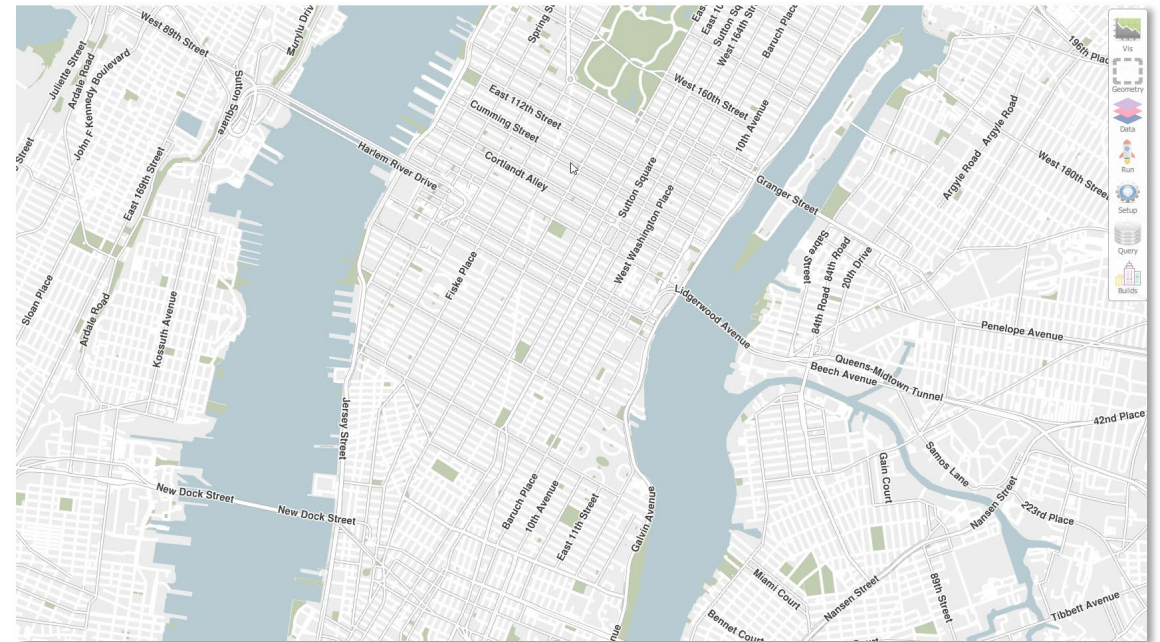
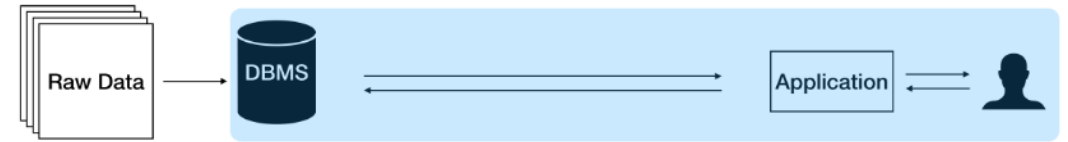


Figure 2: Raster Join first renders all points onto an FBO storing the count of points in each pixel (a). It then aggregates the pixel values corresponding to polygon fragments (b).

Doraiswamy et al., 2018

Backend computation: Database

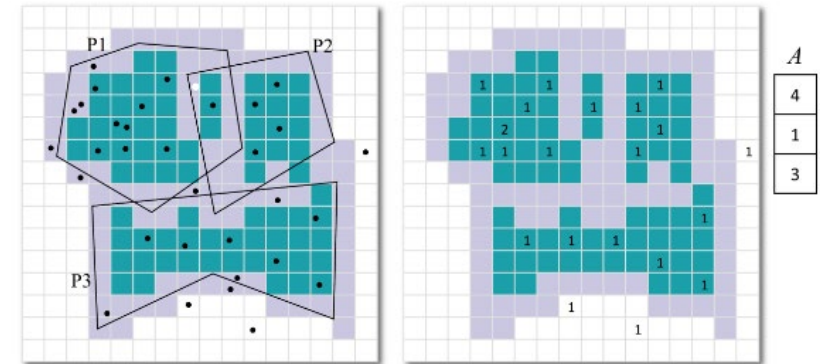
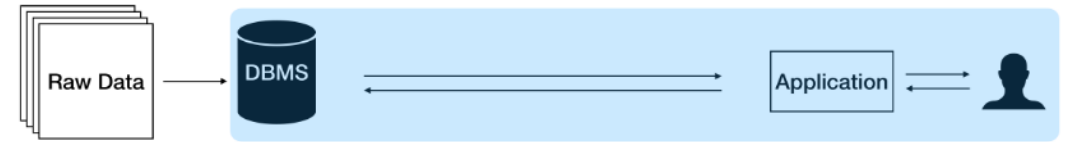
- Raster Join:
 - Reframes spatial aggregation (points-in-polygon + group-by) as a drawing points and polygons on the same GPU canvas, then combining join + aggregation.
 - Rasterize points into an FBO, rasterize polygons / pixels then accumulate.



Doraiswamy et al., 2018

Backend computation: Database

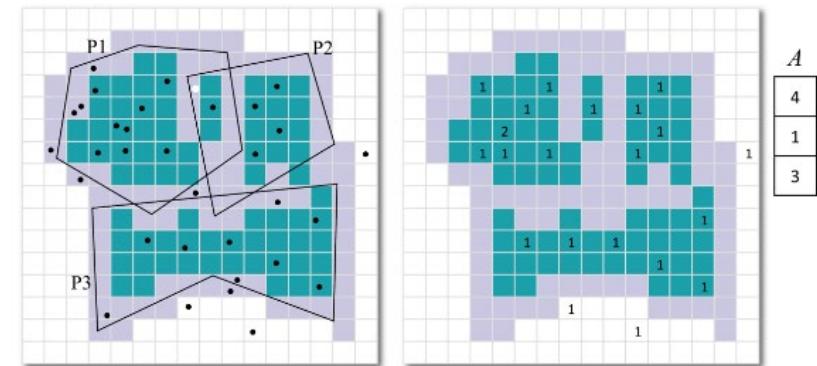
- Raster Join:
 - On real data (geo-tagged Twitter ~2.2B points), delivers:
 - >2 orders of magnitude speedup vs. optimized CPU when in-memory;
 - >30x speedup when not;
 - It can process ~868M points in ~1.1s on a laptop GPU.



Raster join, Tzirita Zacharatou, Doraiswamy et al., 2018

Backend computation: Database

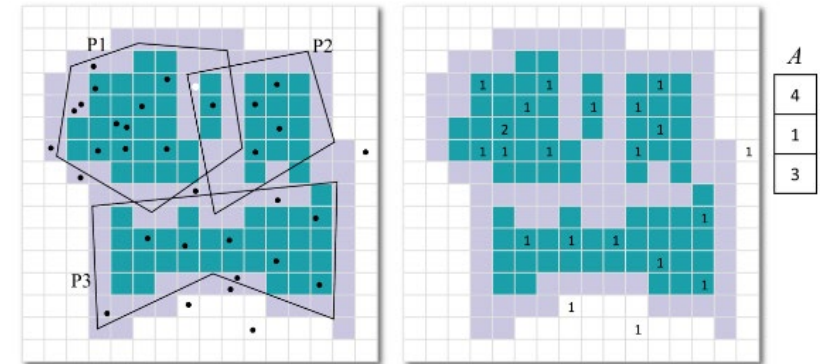
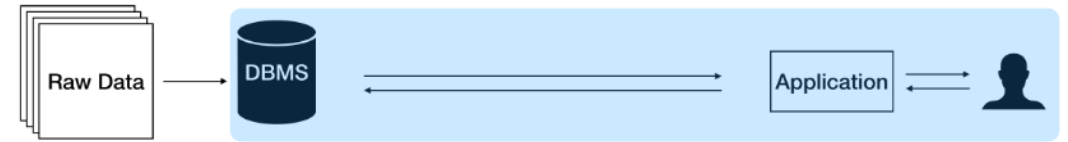
- DB-based approach:
 - Run interactive queries directly in the DB/engine: GROUP BY, filters, spatial predicates, windowed time ranges.
 - Flexibility: supports ad hoc slicing, joins with external tables, and evolving analysis questions.
 - Performance comes from engineering: indexes (spatial/temporal), column stores, partitioning, materialized views.
 - Latency control techniques: query/result caching keyed by viewport+filters; pre-aggregations for “hot” interactions.



Raster join, Tzirita Zacharatou, Doraiswamy et al., 2018

Backend computation: Database

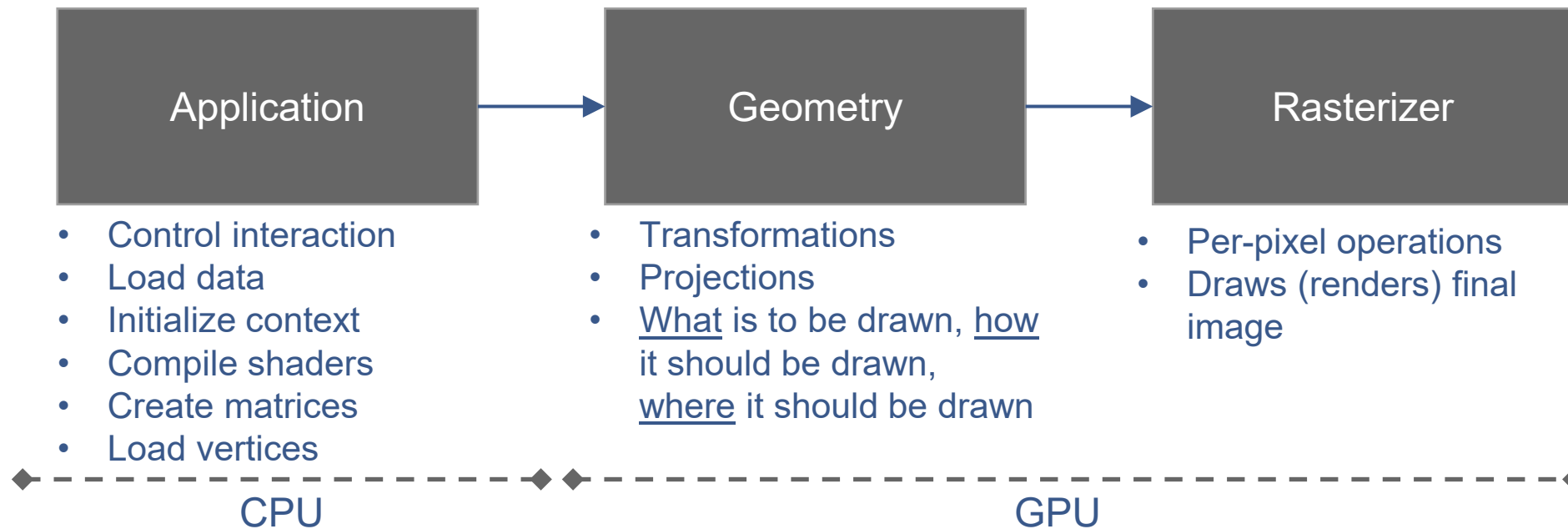
- DB-based approach:
 - **Tradeoffs:** tuning is ongoing; worst-case queries can spike latency if schemas/indexes don't match interactions.
 - **Cost:** if the query can't fully use an index (or needs a new grouping), the DB recomputes aggregates instead of doing $O(1)$ cube lookups.



Raster join, Tzirita Zacharatou, Doraiswamy et al., 2018

Frontend computation: WebGL

- WebGL: GPU rendering for interactive visualizations
 - Graphics library to use the GPU to accelerate the execution of computations in the browser.
 - Primarily known for supporting *rendering* in the browser.
 - Computations are carried through graphics-oriented shaders.



Frontend computation: WebGPU

- WebGPU: GPU pipeline + compute on the web
 - Newer API for general-purpose computing.
 - Computations are carried out through compute-oriented shaders.
 - Supports compute shaders and storage buffers.
 - Compute shaders run in a compute pipeline (separate from render pipeline).
 - WebGPU can support:
 - Draw triangles / points / lines just like WebGL.
 - Run computations on the GPU.

```
const module = device.createShaderModule({ code: wgs1Code });
const pipeline = device.createComputePipeline({
  layout: "auto",
  compute: {
    module,
    entryPoint: "main",
  },
});
```



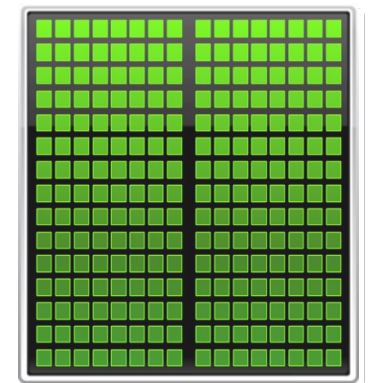
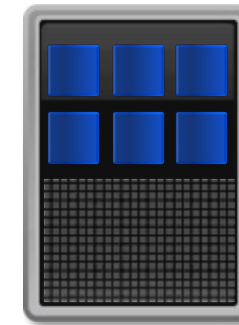
GPU

Accelerator

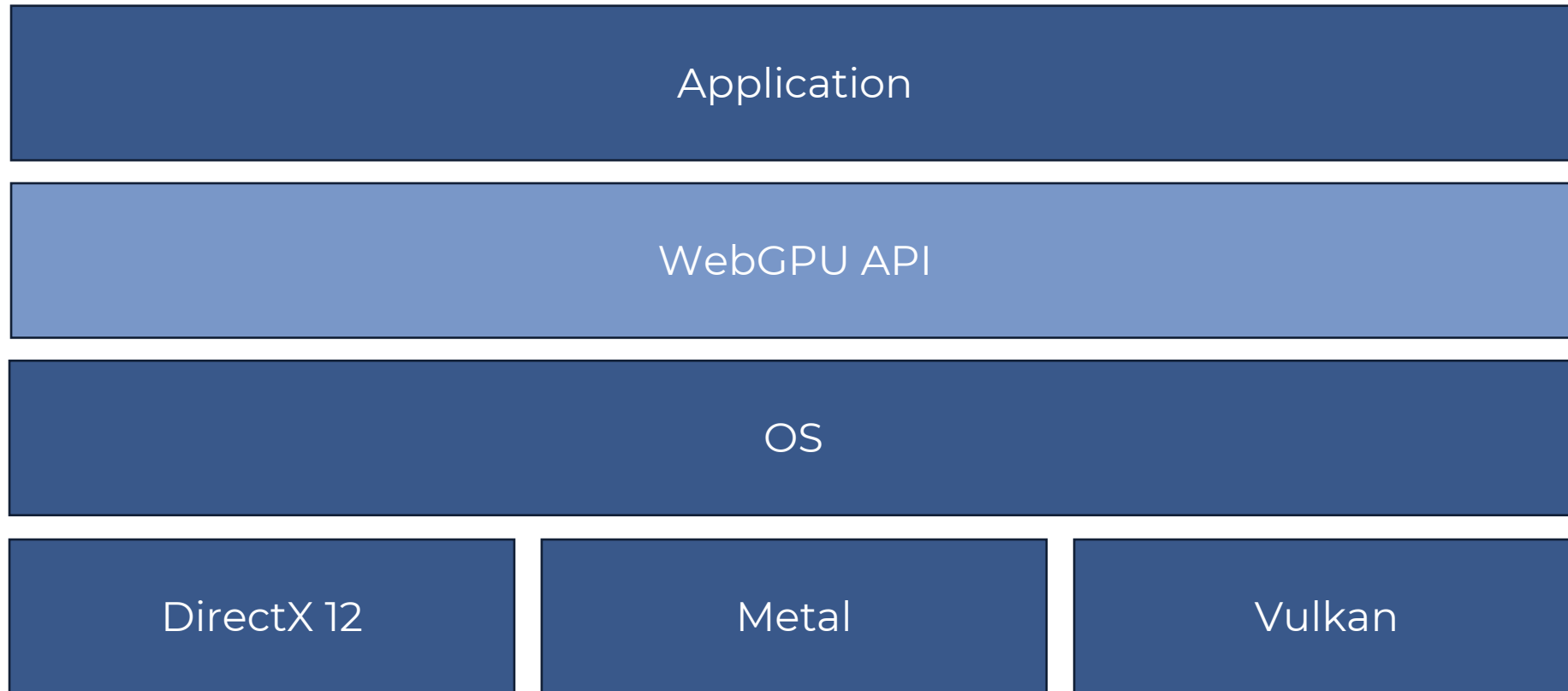
Optimized for Many
Parallel Tasks

CPU

Optimized for
Serial Tasks

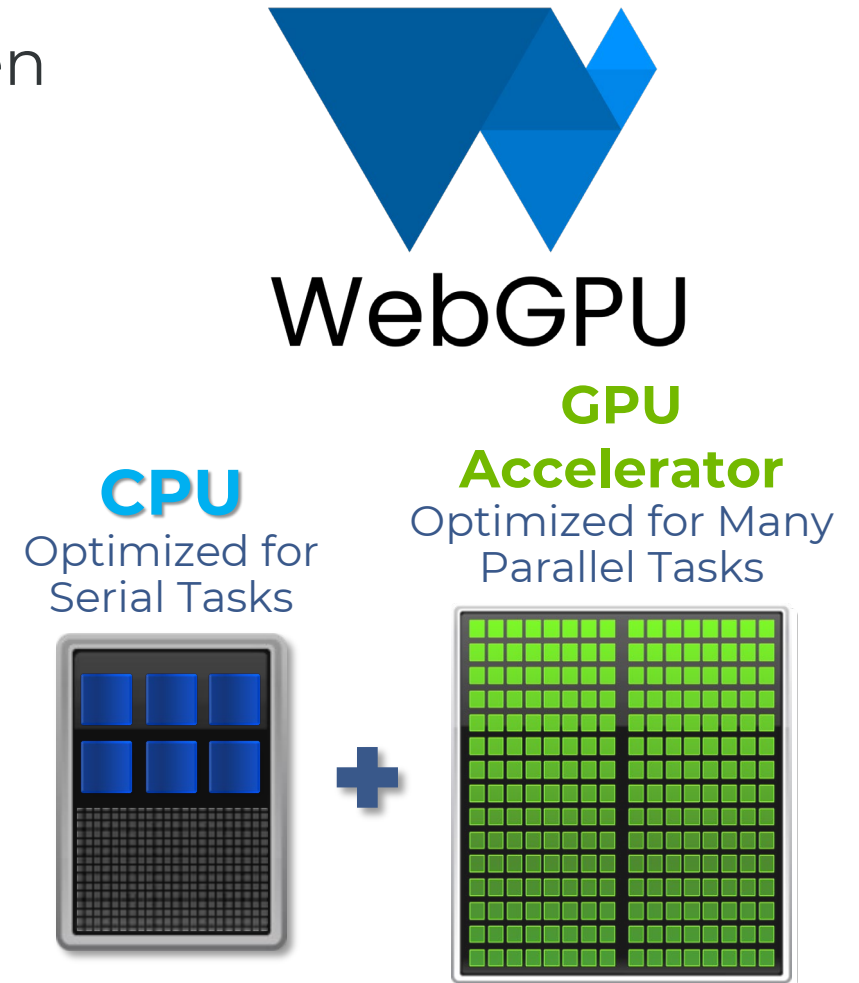


Frontend computation: WebGPU



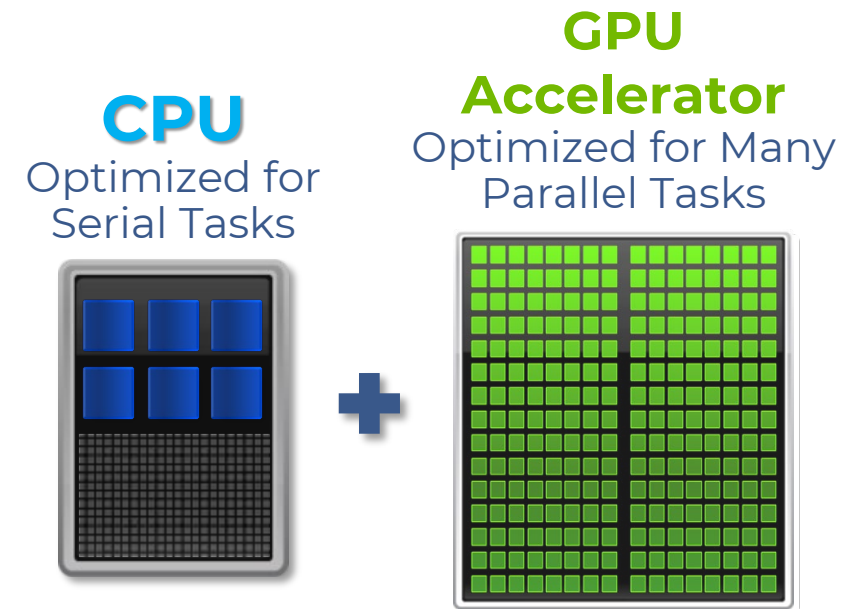
Frontend computation: WebGPU

- WebGL: A state machine where the programmer changes global states and then call draw.
- WebGPU: Explicit pipelines where the programmer creates objects that bundle shader + fixed state. A pipeline bundles:
 - Which shaders should run.
 - Fixed-function state.



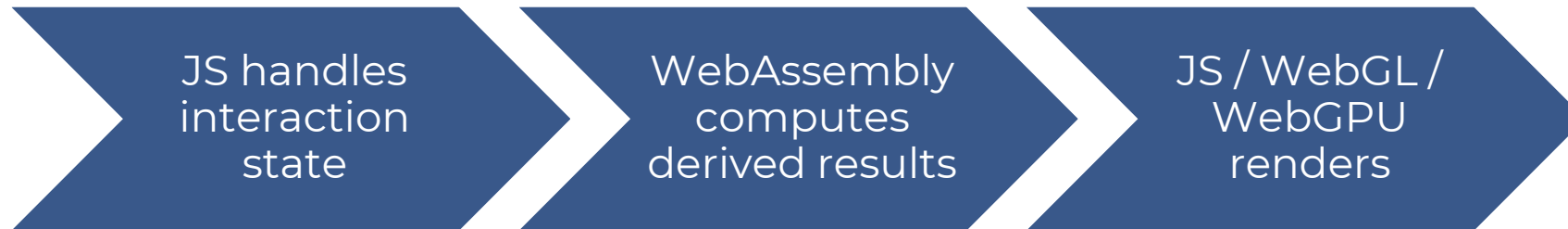
Frontend computation: WebGPU

- Concrete example with blending + depth:
- In WebGL, blending and depth are toggled dynamically (`gl.enable`, `gl.blendFunc`, `gl.enable`).
- In WebGPU, those are part of the render pipeline descriptor: `pipelineOpaque`, `pipelineTransparent`.



Frontend computation: WebAssembly

- WebAssembly: Client-side low-level bytecode
 - Lets the browser run compiled code (C, C++, Rust, etc.).
 - Great for tight loops that dominate latency (e.g., parsing, filtering, clustering, spatial indexing).
 - JS still orchestrates UI + events, WebAssembly is then the **computing engine** you call from JS.
 - Typical pattern:



Frontend computation: WebAssembly

- WebAssembly use cases:
 - Graphics / rendering
 - Visualization
 - Simulation
 - Image & video editing
 - CAD
 - VPN
 - Encryption
 - ...



Frontend computation: WebAssembly

```
extern "C" {  
    int add(int a, int b) { return a + b; }  
}
```

C++

```
em++ add.cpp -O3 \  
-s WASM=1 \  
-s MODULARIZE=1 \  
-s EXPORT_ES6=1 \  
-s EXPORTED_FUNCTIONS='["_add"]' \  
-o add.js
```

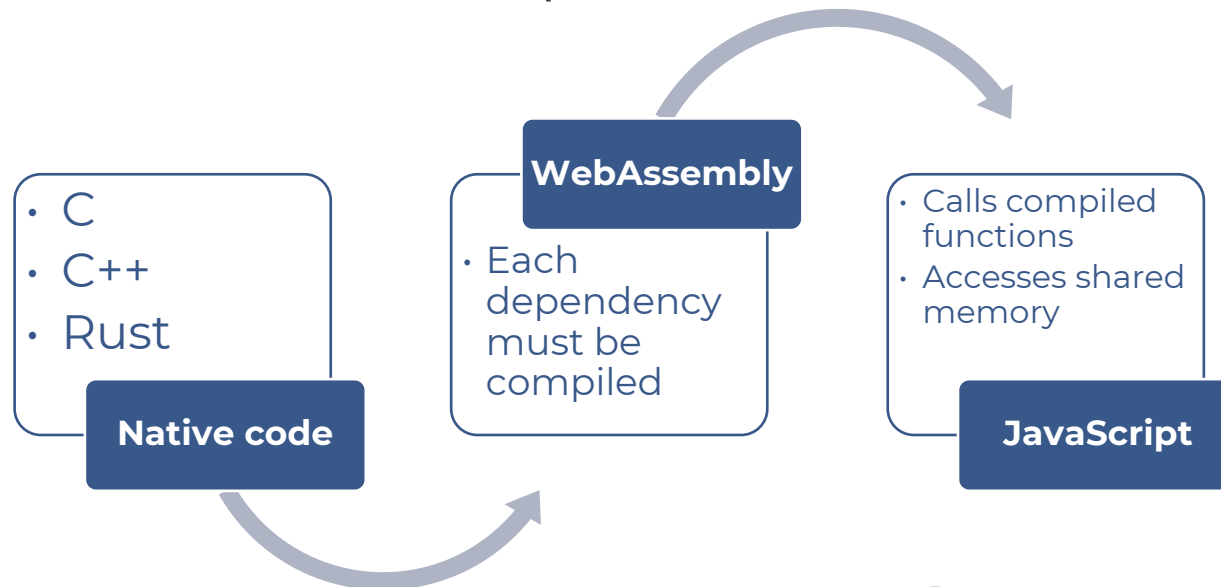
Emscripten compilation

```
<!doctype html>  
<html>  
  <body>  
    <script type="module">  
      import createModule from "./add.js";  
  
      const Module = await createModule();  
      // Emscripten exports C functions with a leading underscore  
      const result = Module._add(40, 2);  
      console.log("add(40,2) =", result);  
    </script>  
  </body>  
</html>
```

HTML / JS to load + call

Real performance comes from a layered stack

- An experimental assessment of WebAssembly and WebGPU as tools for building scalable scientific visualization applications.
- Workflow for interactive isosurfacing of a ~1TB dataset (WebAssembly + WebGPU + compression + GPU cache).



User and Pascucci, 2020

2020 IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV)

Interactive Visualization of Terascale Data in the Browser: Fact or Fiction?

Will Usher* Valerio Pascucci
SCI Institute, University of Utah

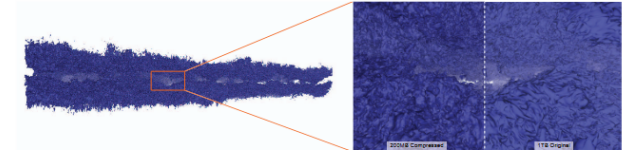


Figure 1: Interactive visualization of an isosurface of a ~1TB dataset entirely in the web browser. The full data is a float64 10240×7680×1536 grid computed by a DNS simulation [20,21]. The isosurface is interactively computed and visualized entirely in the browser using our GPU isosurface computation algorithm for block-compressed data, after applying advanced precision and resolution trade-offs [14, 15, 24]. The surface consists of 137.5M triangles and is computed in 526ms on an RTX 2070 using WebGPU in Chrome and rendered at 30FPS at 1280×720. The original surface, shown in the right split image, consists of 4.3B triangles and was computed with VTK's Flying Edges filter [40] on a quad-socket Xeon server in 78s using 1.5TB of memory.

ABSTRACT

Information visualization applications have become ubiquitous, in no small part thanks to the ease of wide distribution and deployment to users enabled by the web browser. Scientific visualization applications, relying on native code libraries and parallel processing, have been less suited to such widespread distribution, as browsers do not provide the required libraries or compute capabilities. In this paper, we revisit this gap in visualization technologies and explore how new web technologies, WebAssembly and WebGPU, can be used to deploy powerful visualization solutions for large-scale scientific data in the browser. In particular, we evaluate the programming effort required to bring scientific visualization applications to the browser through these technologies and assess their competitiveness against classic native solutions. As a main example, we present a new GPU-driven isosurface extraction method for block-compressed data sets, that is suitable for interactive isosurface computation on large volumes in resource-constrained environments, such as the browser. We conclude that web browsers are on the verge of becoming a competitive platform for even the most demanding scientific visualization tasks, such as interactive visualization of isosurfaces from a 1TB DNS simulation. We call on researchers and developers to consider investing in a community software stack to ease use of these upcoming browser features to bring accessible scientific visualization to the browser.

Index Terms: Web Applications; Parallel Isosurface Extraction

1 INTRODUCTION

Information visualization applications using D3 [7], Vega [37], Vega-Lite [36], and Tableau (formerly Polaris) [43] have become ubiquitous on the web, reaching millions of users to make data more accessible and understandable. The web browser platform is key to enabling this ubiquity, by providing a standardized, cross-platform environment through which applications can be deployed to users.

*will@sci.utah.edu

Similarly, the browser eases the process by which users access an application and enhances security, removing the need to install and trust additional packages on their system. Although the browser has been widely used to deploy information visualization applications, this has not been the case for scientific visualization applications.

Scientific visualization applications often rely on native libraries (e.g., VTK [39]), fast floating point operations, large amounts of memory, and parallel processing or high-performance rendering on the GPU or CPU. Porting the required native libraries and rewriting the application to JavaScript is a significant undertaking, and does not address the computational demands of the application. As a result, the majority of web-based scientific visualization applications rely on a backend server to perform data processing and visualization (e.g., [12, 19, 33–35]), or to stream reduced representations of the data to reduce the demands placed on the browser (e.g., [16, 29, 41, 42]). Using a server gives the application access to significant computational resources, at the cost of latency for users and financial cost to the developer to run the servers. When considering a large-scale deployment, the cost of provisioning sufficient servers to meet demand is a real concern.

WebAssembly [4] and WebGPU [5] are new web technologies that can help address the issues faced when bringing scientific visualization applications to the browser. WebAssembly is a bytecode format to which native code can be compiled (e.g., through Emscripten [45]) and that can be executed at near-native speeds in the browser. WebGPU is a new low-level graphics API for the browser, similar in spirit to DirectX 12, Vulkan, and Metal, although it provides an easier-to-use API. WebGPU exposes advanced rendering and compute functionality to significantly expand the GPU capabilities available to browser applications compared to WebGL. WebGPU supports compute shaders and storage buffers, enabling arbitrary parallel computation on large data, and easing the implementation of such computation compared to prior efforts, which leveraged the WebGL rendering pipeline [22, 23].

In this paper, we perform an initial evaluation of how these new technologies can be leveraged to enable scientific visualization applications to run directly in the browser. We evaluate the performance of widely used native libraries for importing images and LIDAR data when compiled to WebAssembly, and the performance of Marching Cubes [27] as proxy for common floating point and memory

Real performance comes from a layered stack

```
struct LASFile {
    LASreader *reader;
    std::vector<float> positions;
    LASFile(const char *fname); // Reads file using LAStools
};
extern "C" LASFile* openLAS(const char *fname) {
    return new LASFile(fname);
}
extern "C" float* getPositions(LASFile *file){
    return file->positions.data();
}
extern "C" uint64_t getNumPoints(LASFile *file) {
    return file->positions.size() / 3;
}
```

```
// Setup call information for the exported C functions
var openLAS = Module.cwrap("openLAS", "number", ["string"]);
var getPositions =
    Module.cwrap("getPositions", "number", ["number"]);
var getNumPoints =
    Module.cwrap("getNumPoints", "number", ["number"]);
// Write file data into Emscripten's virtual filesystem
FS.writeFile("data.laz", new Uint8Array(fileData));
var lasFile = openLAS("data.laz");
// Create a view of the Wasm module's point data
var positions = new Float32Array(HEAPF32.buffer,
    getPositions(lasFile), getNumPoints(lasFile) * 3);
```

Listing 1: A subset of our C API (top) for the LASlib WebAssembly module and its use from JavaScript (bottom). The C API manages loading the data using LASlib and stores it in memory that can be shared with JavaScript. JS creates a view of the module's memory starting at the "pointer" returned by the API to access the data.

User and Pascucci, 2020

2020 IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV)

Interactive Visualization of Terascale Data in the Browser: Fact or Fiction?

Will Usher* Valerio Pascucci
SCI Institute, University of Utah

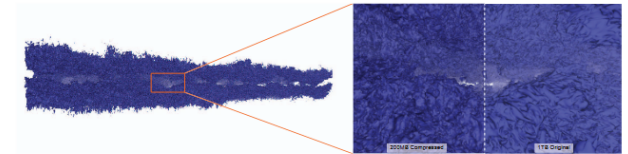


Figure 1: Interactive visualization of an isosurface of a ~1TB dataset entirely in the web browser. The full data is a float64 10240×7680×1536 grid computed by a DNS simulation [20,21]. The isosurface is interactively computed and visualized entirely in the browser using our GPU isosurface computation algorithm for block-compressed data, after applying advanced precision and resolution trade-offs [14, 15, 24]. The surface consists of 137.5M triangles and is computed in 526ms on an RTX 2070 using WebGPU in Chrome and rendered at 30FPS at 1280×720. The original surface, shown in the right split image, consists of 4.3B triangles and was computed with VTK's Flying Edges filter [40] on a quad-socket Xeon server in 78s using 1.5TB of memory.

ABSTRACT

Information visualization applications have become ubiquitous, in no small part thanks to the ease of wide distribution and deployment to users enabled by the web browser. Scientific visualization applications, relying on native code libraries and parallel processing, have been less suited to such widespread distribution, as browsers do not provide the required libraries or compute capabilities. In this paper, we revisit this gap in visualization technologies and explore how new web technologies, WebAssembly and WebGPU, can be used to deploy powerful visualization solutions for large-scale scientific data in the browser. In particular, we evaluate the programming effort required to bring scientific visualization applications to the browser through these technologies and assess their competitiveness against classic native solutions. As a main example, we present a new GPU-driven isosurface extraction method for block-compressed data sets, that is suitable for interactive isosurface computation on large volumes in resource-constrained environments, such as the browser. We conclude that web browsers are on the verge of becoming a competitive platform for even the most demanding scientific visualization tasks, such as interactive visualization of isosurfaces from a 1TB DNS simulation. We call on researchers and developers to consider investing in a community software stack to ease use of these upcoming browser features to bring accessible scientific visualization to the browser.

Index Terms: Web Applications; Parallel Isosurface Extraction

1 INTRODUCTION

Information visualization applications using D3 [7], Vega [37], Vega-Lite [36], and Tableau (formerly Polaris) [43] have become ubiquitous on the web, reaching millions of users to make data more accessible and understandable. The web browser platform is key to enabling this ubiquity, by providing a standardized, cross-platform environment through which applications can be deployed to users.

*will@sci.utah.edu

Similarly, the browser eases the process by which users access an application and enhances security, removing the need to install and trust additional packages on their system. Although the browser has been widely used to deploy information visualization applications, this has not been the case for scientific visualization applications.

Scientific visualization applications often rely on native libraries (e.g., VTK [39]), fast floating point operations, large amounts of memory, and parallel processing or high-performance rendering on the GPU or CPU. Porting the required native libraries and rewriting the application to JavaScript is a significant undertaking, and does not address the computational demands of the application. As a result, the majority of web-based scientific visualization applications rely on a backend server to perform data processing and visualization (e.g., [12, 19, 33–35]), or to stream reduced representations of the data to reduce the demands placed on the browser (e.g., [16, 29, 41, 42]). Using a server gives the application access to significant computational resources, at the cost of latency for users and financial cost to the developer to run the servers. When considering a large-scale deployment, the cost of provisioning sufficient servers to meet demand is a real concern.

WebAssembly [4] and WebGPU [5] are new web technologies that can help address the issues faced when bringing scientific visualization applications to the browser. WebAssembly is a bytecode format to which native code can be compiled (e.g., through Emscripten [45]) and that can be executed at near-native speeds in the browser. WebGPU is a new low-level graphics API for the browser, similar in spirit to DirectX 12, Vulkan, and Metal, although it provides an easier-to-use API. WebGPU exposes advanced rendering and compute functionality to significantly expand the GPU capabilities available to browser applications compared to WebGL. WebGPU supports compute shaders and storage buffers, enabling arbitrary parallel computation on large data, and easing the implementation of such computation compared to prior efforts, which leveraged the WebGL rendering pipeline [22, 23].

In this paper, we perform an initial evaluation of how these new technologies can be leveraged to enable scientific visualization applications to run directly in the browser. We evaluate the performance of widely used native libraries for importing images and LiDAR data when compiled to WebAssembly, and the performance of Marching Cubes [27] as proxy for common floating point and memory

Real performance comes from a layered stack

User and Pascucci, 2020

2020 IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV)

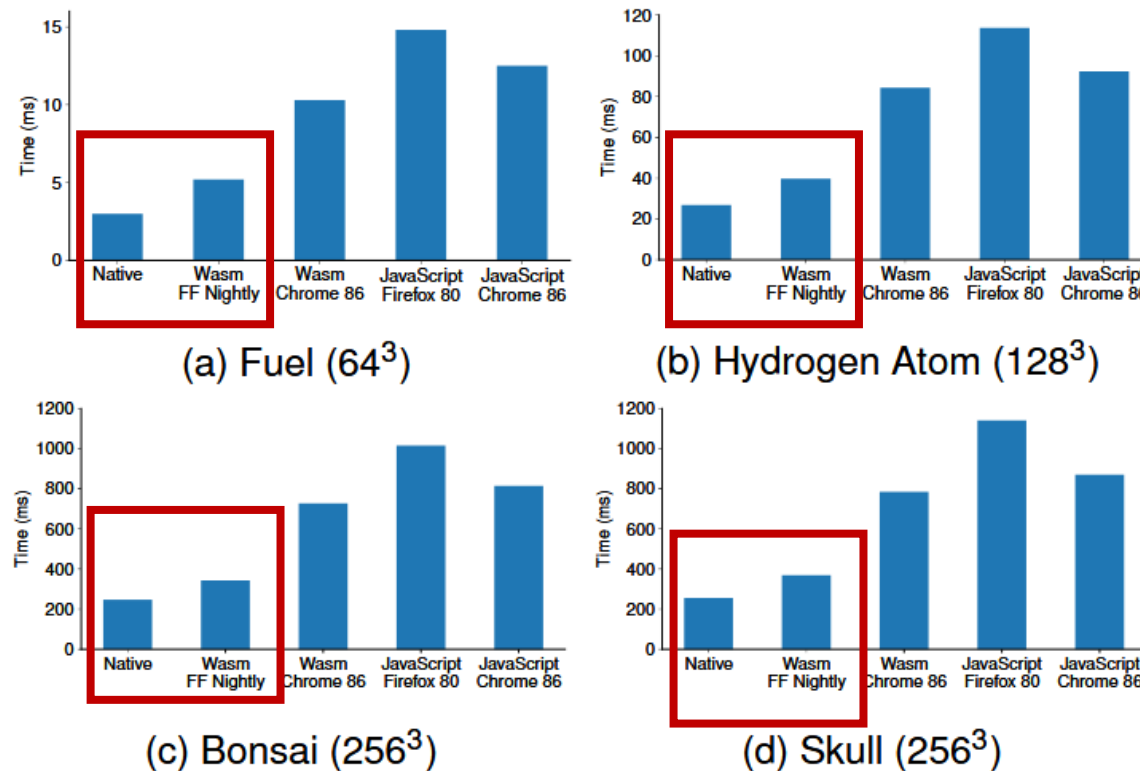


Figure 4: Performance of our serial C++, JavaScript, and WebAssembly versions of Marching Cubes on 100 random isovalues.

Interactive Visualization of Terascale Data in the Browser: Fact or Fiction?

Will Usher* Valerio Pascucci
SCI Institute, University of Utah

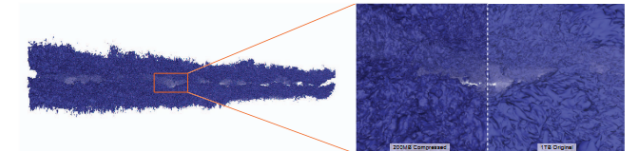


Figure 1: Interactive visualization of an isosurface of a ~1TB dataset entirely in the web browser. The full data is a float64 $10240 \times 7680 \times 1536$ grid computed by a DNS simulation [20,21]. The isosurface is interactively computed and visualized entirely in the browser using our GPU isosurface computation algorithm for block-compressed data, after applying advanced precision and resolution trade-offs [14, 15, 24]. The surface consists of 137.5M triangles and is computed in 526ms on an RTX 2070 using WebGPU in Chrome and rendered at 30FPS at 1280×720 . The original surface, shown in the right split image, consists of 4.3B triangles and was computed with VTK's Flying Edges filter [40] on a quad-socket Xeon server in 78s using 1.5TB of memory.

ABSTRACT

Information visualization applications have become ubiquitous, in no small part thanks to the ease of wide distribution and deployment to users enabled by the web browser. Scientific visualization applications, relying on native code libraries and parallel processing, have been less suited to such widespread distribution, as browsers do not provide the required libraries or compute capabilities. In this paper, we revisit this gap in visualization technologies and explore how new web technologies, WebAssembly and WebGPU, can be used to deploy powerful visualization solutions for large-scale scientific data in the browser. In particular, we evaluate the programming effort required to bring scientific visualization applications to the browser through these technologies and assess their competitiveness against classic native solutions. As a main example, we present a new GPU-driven isosurface extraction method for block-compressed data sets, that is suitable for interactive isosurface computation on large volumes in resource-constrained environments, such as the browser. We conclude that web browsers are on the verge of becoming a competitive platform for even the most demanding scientific visualization tasks, such as interactive visualization of isosurfaces from a 1TB DNS simulation. We call on researchers and developers to consider investing in a community software stack to ease use of these upcoming browser features to bring accessible scientific visualization to the browser.

Index Terms: Web Applications; Parallel Isosurface Extraction

1 INTRODUCTION

Information visualization applications using D3 [7], Vega [37], Vega-Lite [36], and Tableau (formerly Polaris) [43] have become ubiquitous on the web, reaching millions of users to make data more accessible and understandable. The web browser platform is key to enabling this ubiquity, by providing a standardized, cross-platform environment through which applications can be deployed to users.

*will@sci.utah.edu

Similarly, the browser eases the process by which users access an application and enhances security, removing the need to install and trust additional packages on their system. Although the browser has been widely used to deploy information visualization applications, this has not been the case for scientific visualization applications.

Scientific visualization applications often rely on native libraries (e.g., VTK [39]), fast floating point operations, large amounts of memory, and parallel processing or high-performance rendering on the GPU or CPU. Porting the required native libraries and rewriting the application to JavaScript is a significant undertaking, and does not address the computational demands of the application. As a result, the majority of web-based scientific visualization applications rely on a backend server to perform data processing and visualization (e.g., [12, 19, 33–35]), or to stream reduced representations of the data to reduce the demands placed on the browser (e.g., [16, 29, 41, 42]). Using a server gives the application access to significant computational resources, at the cost of latency for users and financial cost to the developer to run the servers. When considering a large-scale deployment, the cost of provisioning sufficient servers to meet demand is a real concern.

WebAssembly [4] and WebGPU [5] are new web technologies that can help address the issues faced when bringing scientific visualization applications to the browser. WebAssembly is a bytecode format to which native code can be compiled (e.g., through Emscripten [45]) and that can be executed at near-native speeds in the browser. WebGPU is a new low-level graphics API for the browser, similar in spirit to DirectX 12, Vulkan, and Metal, although it provides an easier-to-use API. WebGPU exposes advanced rendering and compute functionality to significantly expand the GPU capabilities available to browser applications compared to WebGL. WebGPU supports compute shaders and storage buffers, enabling arbitrary parallel computation on large data, and easing the implementation of such computation compared to prior efforts, which leveraged the WebGL rendering pipeline [22, 23].

In this paper, we perform an initial evaluation of how these new technologies can be leveraged to enable scientific visualization applications to run directly in the browser. We evaluate the performance of widely used native libraries for importing images and LiDAR data when compiled to WebAssembly, and the performance of WebAssembly's Marching Cubes [27] as proxy for common floating point and memory

Real performance comes from a layered stack

Sengupt et al., 2025

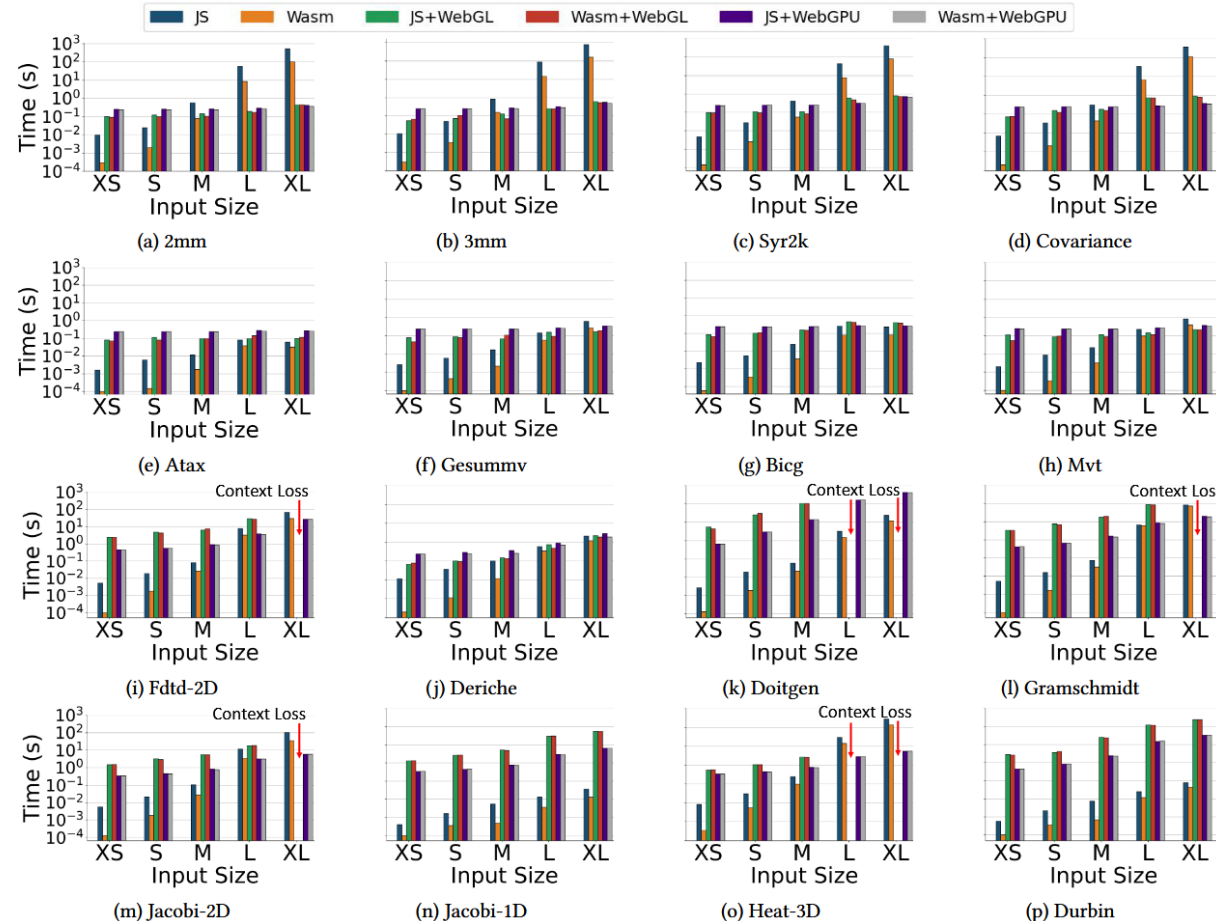


Figure 3: Execution Time for PolyBench Functions with Different Input Data Sizes for Six Configurations.

From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration

Sthitadhi Sengupta
George Mason University
Fairfax, VA, USA
ssengup2@gmu.edu

Krish Jana*
Cornell University
Ithaca, NY, USA
kj364@cornell.edu

Nan Wu
George Mason University
Fairfax, VA, USA
nwu5@gmu.edu

Songqing Chen
George Mason University
Fairfax, VA, USA
sqchen@gmu.edu

Matteo Varvello
Nokia Bell Labs
Murray Hill, NJ, USA
matteo.varvello@nokia.com

Bo Han
George Mason University
Fairfax, VA, USA
bohan@gmu.edu

Abstract

With the rising demand for cost-effective and privacy-preserving deep learning and visualization services, service providers are increasingly turning to in-browser solutions. General-purpose computations, leveraging the graphics processing unit (GPU), are foundational to executing algorithms that power these services. Web graphics library (WebGL) is a widely adopted GPU-access application programming interface (API) designed for multidimensional rendering in browsers, while WebGPU is a newer API developed with compute-specific capabilities. Although WebGPU is a promising standard, its performance has not been systematically evaluated for general-purpose computation. This paper investigates WebGPU and WebGL for accelerating client-side computation in web browsers. By benchmarking key computational GPU kernels for 16 PolyBench and 2 CHStone functions, we measure the performance of WebGPU and WebGL across varying input sizes and algorithmic complexities. Our results show that: 1) both WebGPU and WebGL exhibit poorer performance than central processing unit (CPU)-based execution for small input data due to setup and CPU-GPU synchronization overheads, but they outperform CPU execution as the input data size increases; 2) WebGL performs better than WebGPU for small inputs, except for CPU-driven loop functions, due to its lower initial setup overhead; and 3) WebGPU outperforms WebGL for large inputs through optimized GPU thread utilization and achieves better performance for loop-driven algorithms across all input sizes by minimizing CPU-GPU data exchange. Overall, our results indicate that WebGPU is a competitive option for enhancing the execution performance of large-scale web applications.

CCS Concepts

• Networks → Network measurement; • Information systems → Web applications;

Keywords

Measurement, GPU acceleration, Web performance, WebGPU, WebGL, WebAssembly



This work is licensed under a Creative Commons Attribution 4.0 International License.
IMC '25, Madison, WI, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1860-1/2025/10
<https://doi.org/10.1145/3730567.3764504>

ACM Reference Format:

Sthitadhi Sengupta, Nan Wu, Matteo Varvello, Krish Jana*, Songqing Chen, and Bo Han. 2025. From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. In *Proceedings of the 2025 ACM Internet Measurement Conference (IMC '25)*, October 28–31, 2025, Madison, WI, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3730567.3764504>

1 Introduction

Web applications enable service providers to deliver complex functionality to a broad and diverse set of end-users directly through browsers, ensuring seamless execution on various devices. These applications are becoming increasingly sophisticated, incorporating myriad techniques for image processing and deep learning [26, 29, 42]. General-purpose computations are a fundamental prerequisite for popular image processing and deep learning applications, and they benefit substantially from swift execution in web browsers [42]. Addressing computational bottlenecks is, therefore, crucial to enhancing web application performance and better serving end-users.

Graphics processing unit (GPU)-based acceleration has enhanced the performance of a diverse range of applications in non-web browser settings, such as machine learning, database systems, image processing, and scientific simulations [12, 15, 23, 32]. Effective utilization of the GPU within the client-side web browser can play a key role in alleviating computational bottlenecks while simultaneously reducing server-side costs [26]. In addition, executing applications on the client-side strengthens privacy by eliminating the need to transmit personal data to external servers. Thus, GPU acceleration of general-purpose computations, combined with their client-side execution within the web browser environment, represents an important advancement.

Web graphics library (WebGL) and WebGPU application programming interfaces (APIs) can harness the GPU to accelerate the execution of general-purpose computations in browsers [40]. WebGL is primarily known for enabling multidimensional rendering in browsers [2], whereas WebGPU is a newer API designed to better support general-purpose computations. In WebGL, computations are carried out through *graphics-oriented shaders*, while in WebGPU they are implemented using *compute-oriented shaders* [2, 6–8]. Both WebGL and WebGPU can be programmed in C or C++ and executed on the web by compiling the code to JavaScript or WebAssembly using, for example, the Emscripten compiler [1, 2, 6, 42].

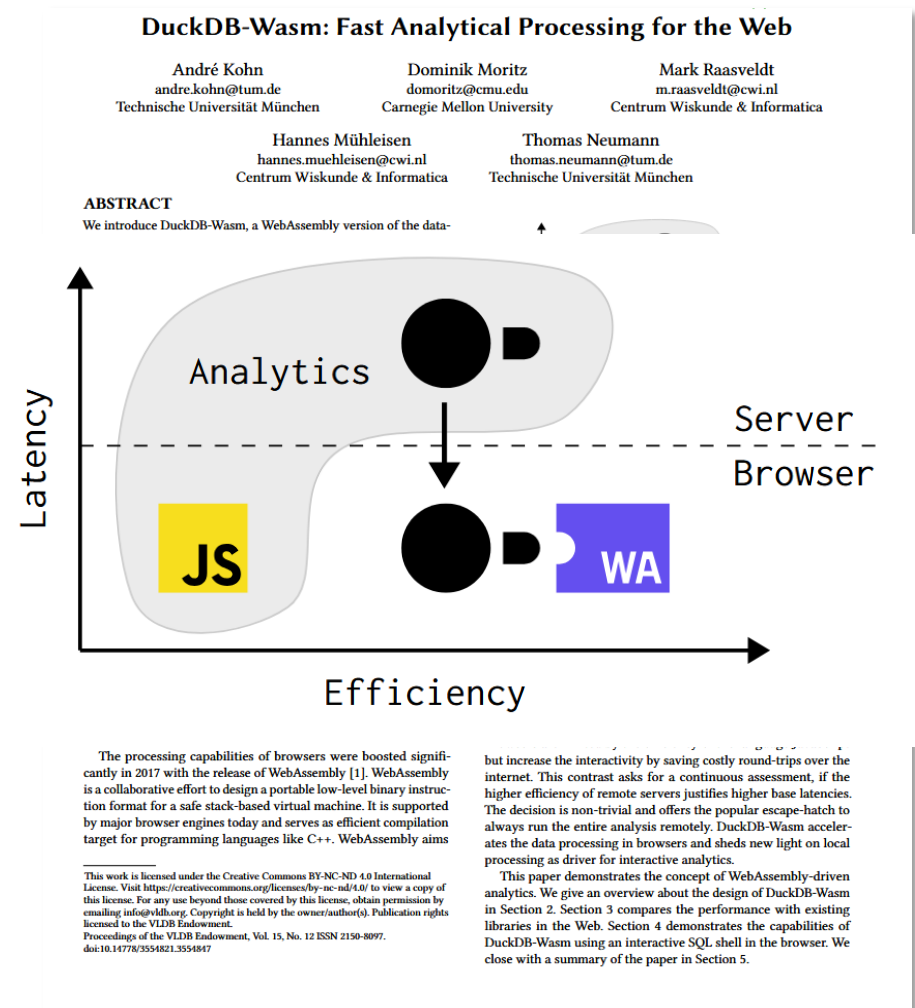
*Work was done when at Montville Township High School.

1018

A full-fledged database built with WebAssembly

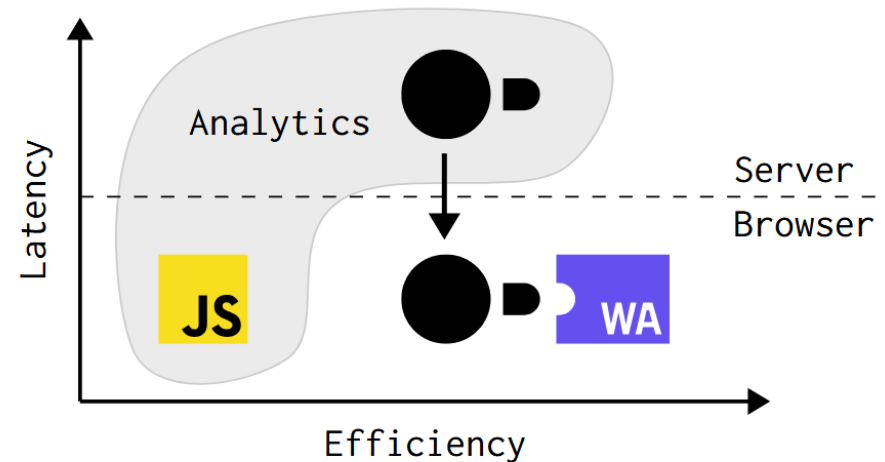
- Browsers are powerful but JavaScript is slow for analytics.
- Usual path: Push data analysis to servers, but round-trip latency hurts interactivity.
- **DuckDB-Wasm: Analytics on the browser.**
- WebAssembly-compiled version of DuckDB.

Kohn et al., 2022



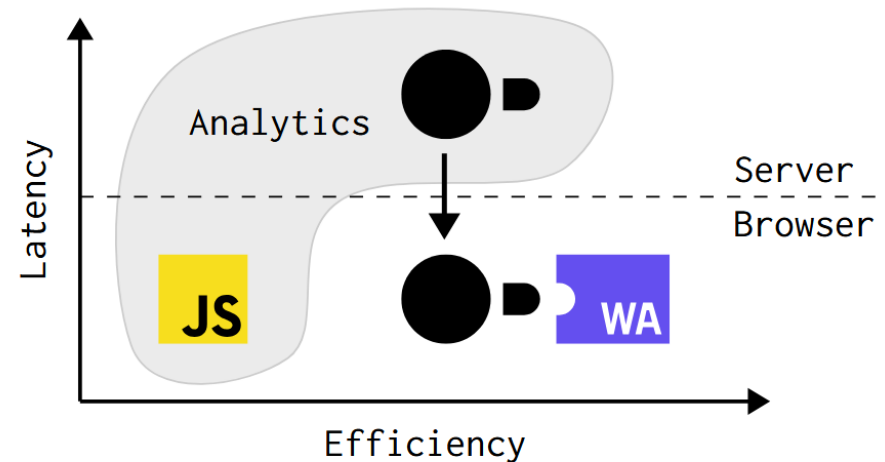
A full-fledged database built with WebAssembly

- Browsers are powerful but JavaScript is slow for analytics.
- Usual path: Push data analysis to servers, but round-trip latency hurts interactivity.
- **DuckDB-Wasm: Analytics on the browser.**
- WebAssembly-compiled version of DuckDB.



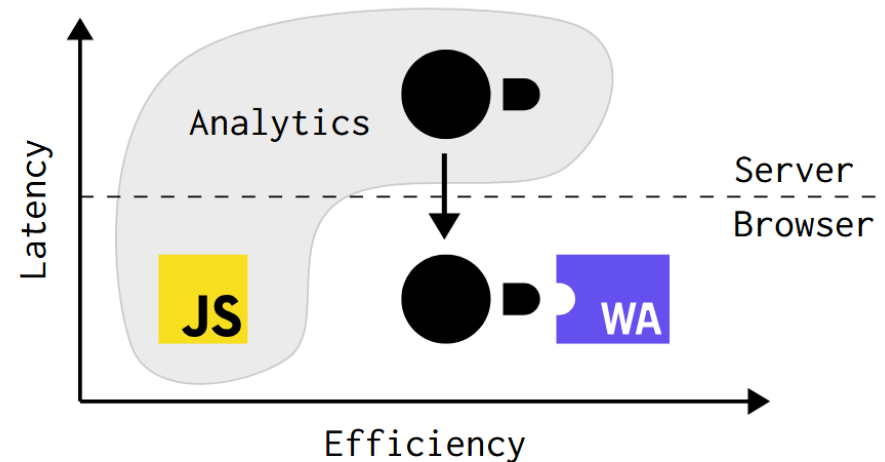
A full-fledged database built with WebAssembly

- DuckDB doesn't process rows one-by-one, it processes data in chunks (~1024 rows at a time).
- Operators (scan, filter, join, aggregate, etc.) all consume and produce *chunks*, not single *rows*.



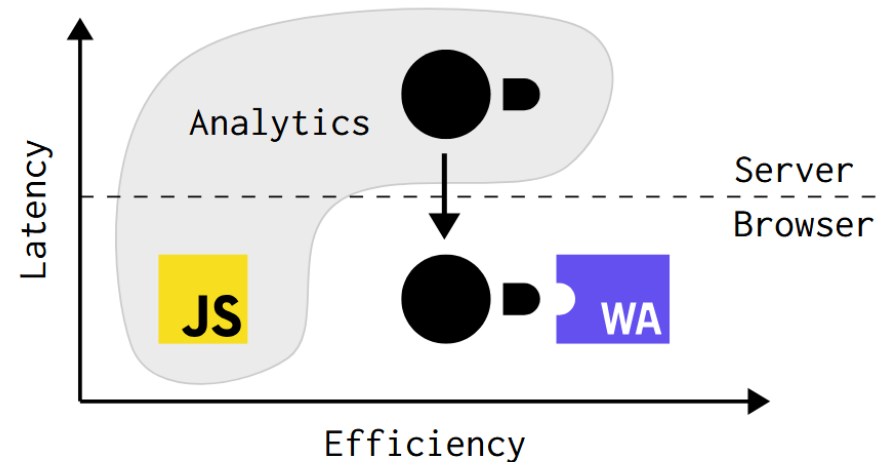
A full-fledged database built with WebAssembly

- DuckDB doesn't process rows one-by-one, it processes data in chunks (~1024 rows at a time).
- Operators (scan, filter, join, aggregate, etc.) all consume and produce *chunks*, not single *rows*.
- Why that's fast?
 - **Amortizes overhead:** queries are not repeated per row.
 - **Runs tight loops:** predictable loops over contiguous arrays.
 - **Improve cache locality:** sequential memory access is cache-friendly.
 - **Enables compiler / CPU tricks:** auto-vectorization, SIMD.



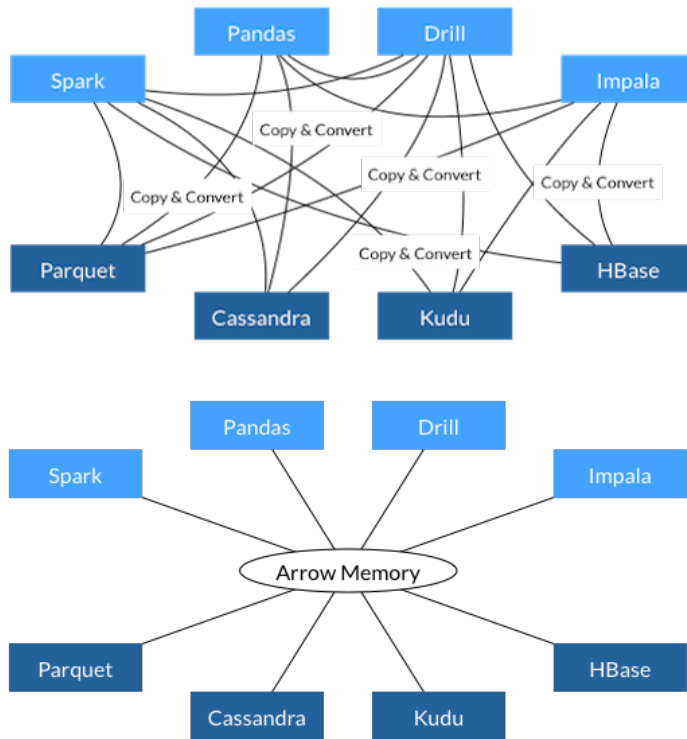
A full-fledged database built with WebAssembly

- Native DuckDB: vectors live in normal process memory.
- DuckDB-Wasm: vectors live in WebAssembly linear memory.
- Same logic: “here is a pointer to the column data, loop over it.”

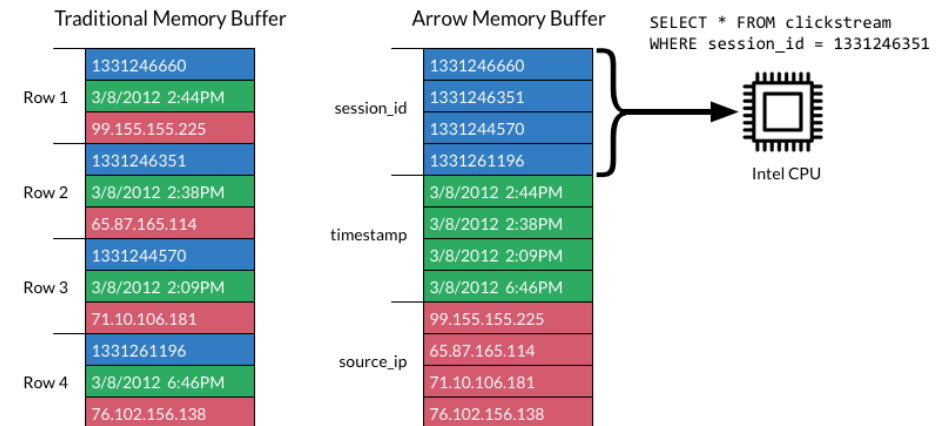


A full-fledged database built with WebAssembly

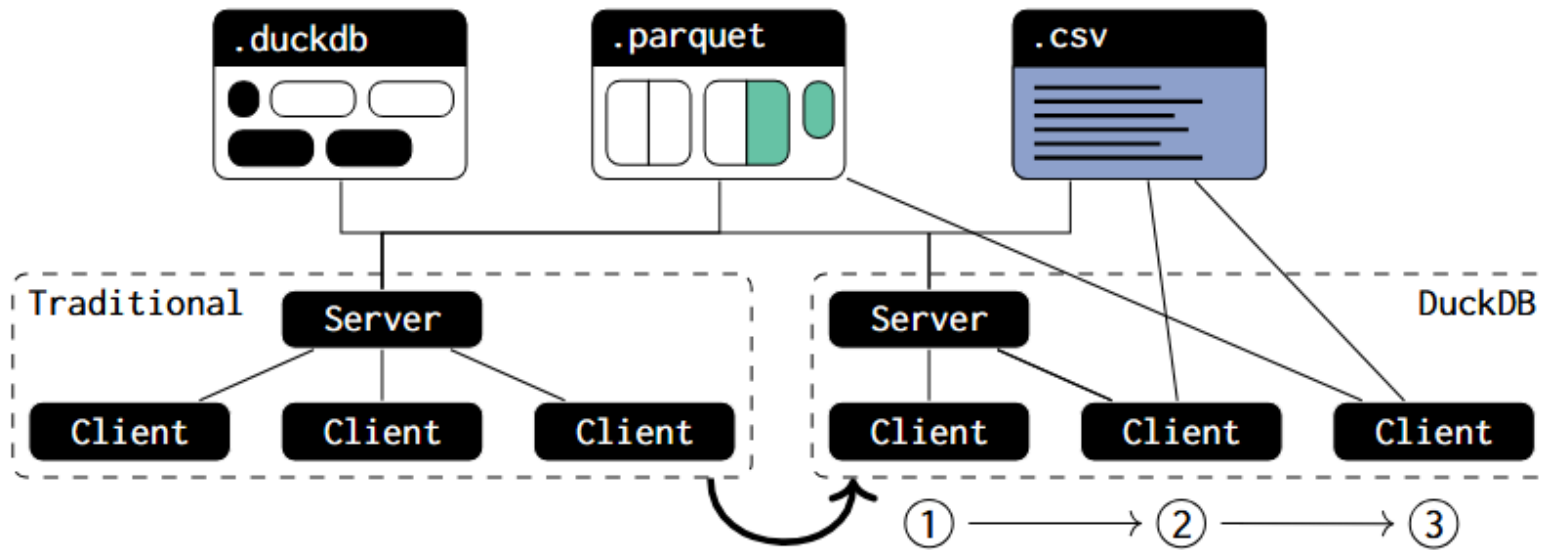
- Uses Apache Arrow as its in-memory columnar format.
- Language-agnostic representation for structured data.



	session_id	timestamp	source_ip
Row 1	1331246660	3/8/2012 2:44PM	99.155.155.225
Row 2	1331246351	3/8/2012 2:38PM	65.87.165.114
Row 3	1331244570	3/8/2012 2:09PM	71.10.106.181
Row 4	1331261196	3/8/2012 6:46PM	76.102.156.138



A full-fledged database built with WebAssembly



Traditional approach:

Computation is done by dedicated servers.

DuckDB-Wasm:

(1) Computation is done by dedicated servers.

(3) Aggregation and filtering using a server, joining on the local device.

(2) Analytics run locally.

```
CREATE VIEW recentdaily AS (  
  SELECT date_trunc('day', ts) AS day,  
         name, max(price) AS price  
  FROM 's3://bucket/stocks.parquet'  
  WHERE ts > current_date - 30  
  GROUP BY day);  
  
SELECT r.day, p.name, max(p.count*r.price)  
  FROM 'https://remote/portfolio.csv' p,  
       recentdaily r  
 WHERE p.name = r.name GROUP BY ALL;
```

A full-fledged database built with WebAssembly

- User-defined functions:
 - If you let people call JavaScript functions from SQL, queries can become slow because of the inherent overhead.
 - But: DuckDB doesn't process a query one row at a time; it processes data in chunks.
 - Instead of:

for each row: call JS → return value

- DuckDB-Wasm:

for each chunk: call JS on the chunk's value

```
import * as duckdb from '@duckdb/duckdb-wasm';  
(...)  
conn.createScalarFunction('double_it', new Float64(), (x) => x * 2);  
(...)  
const out = conn.query(`SELECT double_it(21) AS v;`);
```



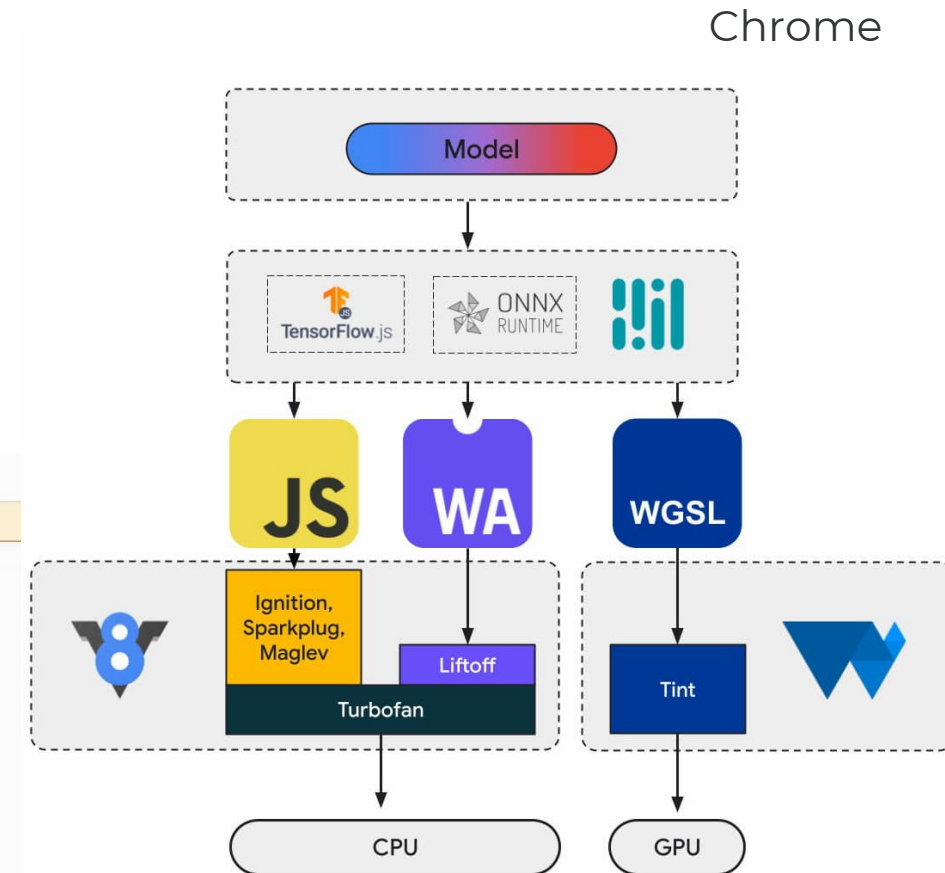
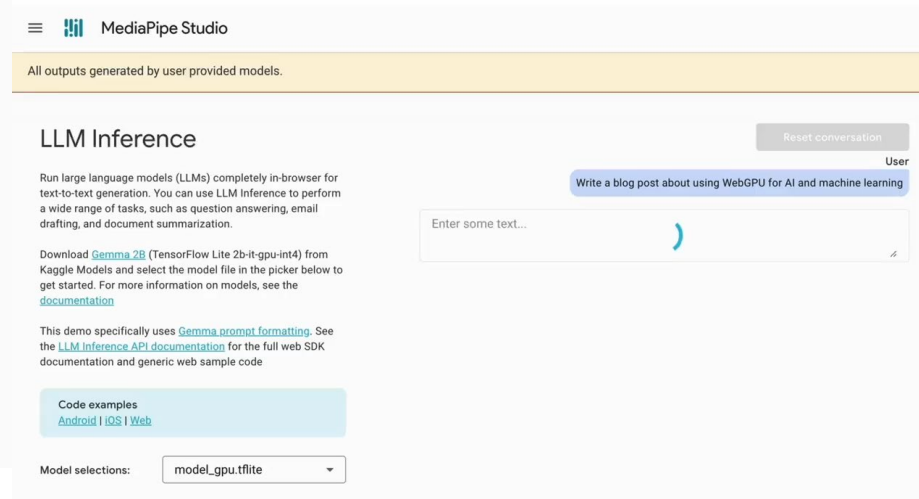
A full-fledged database built with WebAssembly

Table 1: Execution times in seconds for TPC-H queries at the scale factors 0.01, 0.1 and 0.5.

#	SF = 0.01				SF = 0.1				SF = 0.5			
	DuckDB	SQL.js	Arquero	Lovefield	DuckDB	SQL.js	Arquero	Lovefield	DuckDB	SQL.js	Arquero	Lovefield
1	0.005	0.054	0.063	0.046	0.047	0.584	0.823	0.805	0.235	3.412	9.080	4.979
2	0.002	0.002	0.003	—	0.005	0.019	0.122	—	0.015	0.101	3.314	—
3	0.002	0.014	0.047	0.020	0.008	0.150	0.570	0.281	0.048	0.791	5.923	1.626
4	0.001	0.003	0.028	0.014	0.008	0.033	0.361	0.234	0.048	0.181	2.060	1.573
5	0.003	0.013	0.020	0.008	0.008	0.153	0.539	0.178	0.049	0.875	9.498	1.415
6	0.001	0.010	0.009	0.007	0.005	0.100	0.107	0.121	0.026	0.532	0.622	0.793
7	0.003	0.017	0.049	0.016	0.016	0.202	0.491	0.498	0.086	1.174	2.574	12.391
8	0.004	0.020	0.016	0.008	0.010	0.288	0.157	0.189	0.070	1.831	0.894	1.399
9	0.006	0.027	0.716	0.211	0.057	0.481	—	3.243	0.483	3.317	—	—
10	0.003	0.010	0.029	0.013	0.020	0.106	0.420	0.270	0.116	0.559	7.784	1.678
11	0.001	0.004	0.001	—	0.003	0.050	0.007	—	0.007	0.265	0.039	—
12	0.003	0.009	0.012	0.017	0.019	0.096	0.154	0.277	0.089	0.493	1.992	1.704
13	0.002	0.020	0.012	0.044	0.014	0.327	0.197	0.572	0.068	2.246	5.200	3.251
14	0.001	0.009	0.039	0.013	0.005	0.094	0.405	0.223	0.024	0.498	2.261	1.475
$\varnothing_{geo}$	0.003	0.012	0.023	0.019	0.013	0.142	0.268	0.338	0.073	0.809	2.822	2.049

Incredible opportunities

- Pushing workloads to the browser.
 - Inference without requiring a server.
 - “Same” C++ application that can run on-device can also run on the web (with modifications).



Embeddable Opportunities

Demonstration

SIGMOD '19, June 30-July 5, 2019, Amsterdam, Netherlands

DuckDB: an Embeddable Analytical Database

Mark Rausveldt
m.rausveldt@cwi.nl
CWI, Amsterdam

Hannes Mühleisen
hannes.muehleisen@cwi.nl
CWI, Amsterdam

ABSTRACT

The immense popularity of SQLite shows that there is a need for unobtrusive in-process data management solutions. However, there is no such system yet geared towards analytical workloads. We demonstrate DuckDB, a novel data management system designed to execute analytical SQL queries while embedded in another process. In our demonstration, we put DuckDB against other data management solutions to showcase its performance in the embedded analytics scenarios. DuckDB is available as Open Source software under a permissive license.

ACM Reference Format:
Mark Rausveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *2019 International Conference on Management of Data (SIGMOD '19)*, June 30-July 5, 2019, Amsterdam, Netherlands. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3326652.3326712>

1 INTRODUCTION

Data management systems have evolved into large monolithic database servers running as stand-alone processes. This is partly a result of the need to serve requests from many clients simultaneously and partly due to data integrity requirements. While powerful, stand-alone systems require considerable effort to set up properly and data access is constrained by their client protocols [12]. There exists a completely separate use case for data management systems, those that are embedded into other processes where the database system is a linked library that runs completely within a "host" process. The most well-known representative of this group is SQLite, the most widely deployed SQL database engine with more than a trillion databases in active use [4]. SQLite strongly focuses on transactional (OLTP) workloads, and contains a row-major execution engine operating on a B-tree storage format [3]. As a consequence, SQLite's performance on analytical (OLAP) workloads is very poor.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for this part of the work must be honored. For all other uses, contact the owner(s).

ACM ISBN 978-1-4503-5917-7/19/07...\$15.00.
2019 Copyright held by the owner(s).
ACM ISBN 978-1-4503-5917-7/19/07...\$15.00.
<https://doi.org/10.1145/3326652.3326712>

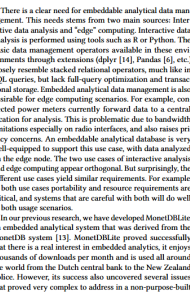


Figure 1: Systems Landscape

There is a clear need for embeddable analytical data management. This needs stems from two main sources: data integration, data analysis and "edge" computing. Interactive data analysis is performed using tools such as R or Python. The basic data management operations available in these environments through extensions (dplyr [14], Pandas [4], etc.) closely resemble stacked relational operators, much like in SQL queries, but lack full-query optimization and transactional storage. Embeddable analytical data management is also desirable for edge computing scenarios. For example, connected power meters currently forward data to a central location for analysis. This is problematic due to bandwidth limitations especially on radio interfaces, and also raises privacy concerns. An embeddable analytical database is very well-equipped to support this use case, with data analyzed on the edge node. The two use cases of interactive analysis and edge computing appear orthogonal, but surprisingly, they share different use cases and similar requirements. For example, in both use cases portability and resource requirements are critical, and systems that are careful with both will do well in both use cases scenarios.

In our previous research, we have developed MonetDB/OLE, an embeddable analytical system that was derived from the MonetDB system [13]. MonetDB/OLE proved successfully that there is a real interest in embeddable analytics; it supports thousands of downloads per month and is used all around the world from the Dutch central bank to the New Zealand police. However, its success also uncovered several issues that proved very complex to address in a non-purpose-built

DuckDB-Wasm: Fast Analytical Processing for the Web

André Kohn
andre.kohn@um.de
Technische Universität München

Dominik Moritz
domoritz@cs.cmu.edu
Carnegie Mellon University

Mark Rausveldt
m.rausveldt@cwi.nl
Centrum Wiskunde & Informatica

Hannes Mühleisen
hannes.muehleisen@cwi.nl
Centrum Wiskunde & Informatica

Thomas Neumann
thomas.neumann@um.de
Technische Universität München

ABSTRACT

We introduce DuckDB-Wasm, a WebAssembly version of the database system DuckDB, to provide fast analytical processing for the Web. DuckDB-Wasm evaluates SQL queries asynchronously in web workers, supports efficient user-defined functions written in JavaScript, and features a browser-agnostic interface that handles local and remote data in pages. DuckDB-Wasm outperforms previous data processing libraries for the Web in the TPC-H benchmarks at multiple scale factors. We demonstrate the capabilities of an analytical database in the browser using an interactive SQL shell.

ACM Reference Format:
André Kohn, Dominik Moritz, Mark Rausveldt, Hannes Mühleisen, and Thomas Neumann. DuckDB-Wasm: Fast Analytical Processing for the Web. *VLDB*, VLDB '19, 2019, 2019. <https://doi.org/10.1145/3326652.3326712>

VLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at <https://github.com/duckdb/duckdb-wasm>

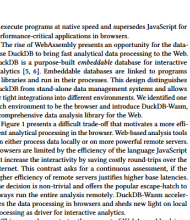
1 INTRODUCTION

The web browser has evolved to a universal computation platform. In the last few years, it has been accompanied by increasing requirements for the browser programming language JavaScript. JavaScript was designed to be flexible which comes at the cost of a reduced processing efficiency. This is pronounced when considering the execution times of complex data analysis tasks that often fall behind the native execution by orders of magnitude. In the past, analysis tools have therefore been pushed to servers that tie any client-side processing to additional round trips over the Internet. These round trips introduce network latencies that negatively affect interactive data exploration [15].

The processing capabilities of browsers were boosted significantly in 2017 with the release of WebAssembly [1]. WebAssembly is a collaborative effort to design a portable low-level binary instruction format for a wide stack-based virtual machine. It is supported by major browser engines today and serves as an efficient compilation target for programming languages like C++ or WebAssembly assembly.

The work is licensed under the Creative Commons Attribution 4.0 International License. For more information, see <https://creativecommons.org/licenses/by/4.0/>.
This paper is part of a book that is part of the SIGMOD '19 proceedings. For more information, see <https://www.sigmod.org/>.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for this part of the work must be honored. For all other uses, contact the owner(s).
ACM ISBN 978-1-4503-5917-7/19/07...\$15.00.
2019 Copyright held by the owner(s).
ACM ISBN 978-1-4503-5917-7/19/07...\$15.00.
<https://doi.org/10.1145/3326652.3326712>

Figure 1: Browser-based analytics tools process data either locally with a low efficiency or on servers with a high latency. DuckDB-Wasm pushes the boundaries with fast analytical processing for the Web.



This paper demonstrates the concept of WebAssembly-driven databases. We give an overview about the design of DuckDB-Wasm in Section 2. Section 3 compares the performance with existing libraries in the Web. Section 4 demonstrates the capabilities of DuckDB-Wasm using an interactive SQL shell in the browser. We close with a summary of the paper in Section 5.

418

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 26, NO. 1, JANUARY 2020

Mosaic: An Architecture for Scalable & Interoperable Data Views

Jeffrey Heer
jeff@heer.org

Dominik Moritz
domoritz@cs.cmu.edu

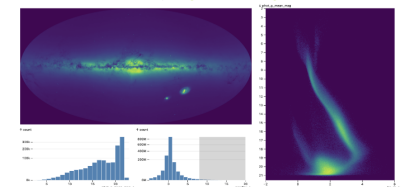


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia star catalog. A high-resolution detail map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a hierarchical nested diagram of color versus stellar magnitude on the right. Mosaic efficiently displays and manages computation in a backing scalable database, and automatically builds optimized data cube indexes to support interactive insight views.

Abstract. Mosaic is an architecture for greater scalability, extensibility, and interoperability of interactive data views. Mosaic decouples data processing from specification logic: clients publish their data needs as declarative queries that are then managed and automatically optimized by a coordinator that proxies access to a scalable data store. Mosaic operations (Vega-Lite selection abstraction) to enable rich integration and tracing across visualizations and components such as menus, text, and tables. We demonstrate Mosaic's responsiveness, extensibility, and interoperability through examples that compare diverse visualizations, interaction, and optimization techniques—many constructed using agave, a grammar of interactive graphics in which graphical marks are not Mosaic clients. To illustrate scalability, we present benchmark studies with order-of-magnitude performance improvements over existing web-based visualization systems—enabling flexible, real-time visual exploration of billions' recent datasets. We conclude by discussing Mosaic's potential as an open platform that bridges visualization languages, scalable visualization, and interactive data systems more broadly.

Index Terms. Visualization, Interaction, Scalability, Grammar of Graphics, Software Architecture, Databases

1 INTRODUCTION

Through many expressive visualization tools exist, scalability to large datasets and interoperability across toolchains remains challenging [1]. The visualization community lacks open, standardized tools for integrating visualizations with scalable analytic databases. While libraries like D3.js [18] enhance Web standards for cross-tool interoperability, high-level frameworks often make closed-world assumptions, complicating integration with other tools and environments.

As a concrete example, consider the Vega [20] and Vega-Lite [21] ecosystems. By default, data transformation are performed within a JavaScript runtime, limiting scalability due to both data movement and a lack of parallel computing. Meanwhile, other architectural decisions impede extensibility and interoperability. Vega-Lite's selection abstraction provides a powerful, concise model for interaction, yet is realized in terms of opaque internal Vega constructs that complicate coordination with external components and environments such as notebooks. As a result, online requests for table views, data-driven input widgets, more interoperable selections, and new mark types (e.g., for raster heatmaps and contour plots in Vega-Lite) have gone unaddressed for years. Other tools face similar limitations [4, 15].

We propose a standardized "middle tier" architecture that mediates data-driven components and backing data stores. A common layer between databases and components can coordinate linked selections and parameter sharing views, while providing automatic query optimizations for greater scalability. We focus on the Web browser as the primary site of rendering and interaction, and seek to coordinate diverse components via standard protocols for communicating data needs, dynamic parameters, and linked selection views. We contribute Mosaic: an architecture for interoperable, data-driven components—including visualizations, tables, and input widgets—linked by scalable data stores. A key idea of Mosaic is to decouple data processing from specification. Mosaic clients communicate their data needs as declarative queries. A central Coordinator manages these queries, applying automatic optimizations, and pushes processing to a data source that handles the data. This architecture is designed to be extensible and interoperable. See <https://github.com/jeffreyheer/mosaic> and <https://github.com/jeffreyheer/mosaic>

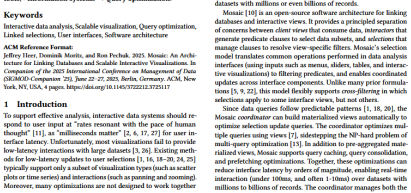
ACM Reference Format:
Jeffrey Heer and Dominik Moritz. 2020. Mosaic: An Architecture for Linking Databases and Scalable Interactive Visualizations. In *2020 International Conference on Management of Data (SIGMOD '20)*, June 20-25, 2020, Portland, Oregon, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3326652.3326712>

VLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at <https://github.com/jeffreyheer/mosaic>

1 INTRODUCTION
The web browser has evolved to a universal computation platform. In the last few years, it has been accompanied by increasing requirements for the browser programming language JavaScript. JavaScript was designed to be flexible which comes at the cost of a reduced processing efficiency. This is pronounced when considering the execution times of complex data analysis tasks that often fall behind the native execution by orders of magnitude. In the past, analysis tools have therefore been pushed to servers that tie any client-side processing to additional round trips over the Internet. These round trips introduce network latencies that negatively affect interactive data exploration [15].

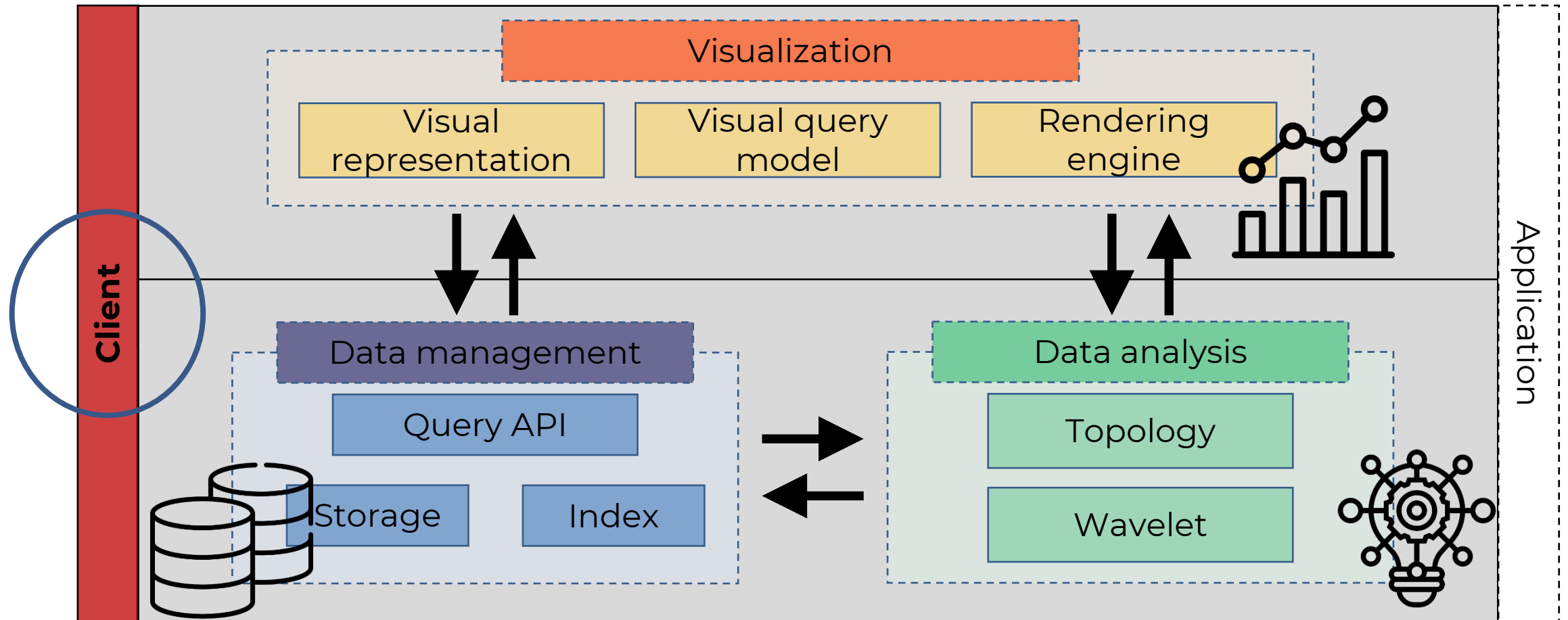
The processing capabilities of browsers were boosted significantly in 2017 with the release of WebAssembly [1]. WebAssembly is a collaborative effort to design a portable low-level binary instruction format for a wide stack-based virtual machine. It is supported by major browser engines today and serves as an efficient compilation target for programming languages like C++ or WebAssembly assembly. The work is licensed under the Creative Commons Attribution 4.0 International License. For more information, see <https://creativecommons.org/licenses/by/4.0/>.
This paper is part of a book that is part of the SIGMOD '20 proceedings. For more information, see <https://www.sigmod.org/>.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for this part of the work must be honored. For all other uses, contact the owner(s).
ACM ISBN 978-1-4503-5917-7/20/07...\$15.00.
2020 Copyright held by the owner(s).
ACM ISBN 978-1-4503-5917-7/20/07...\$15.00.
<https://doi.org/10.1145/3326652.3326712>

Figure 1: Browser-based analytics tools process data either locally with a low efficiency or on servers with a high latency. DuckDB-Wasm pushes the boundaries with fast analytical processing for the Web.



This paper demonstrates the concept of WebAssembly-driven databases. We give an overview about the design of DuckDB-Wasm in Section 2. Section 3 compares the performance with existing libraries in the Web. Section 4 demonstrates the capabilities of DuckDB-Wasm using an interactive SQL shell in the browser. We close with a summary of the paper in Section 5.

Big data visualization system



Main takeaways

- The “web can’t do serious vis” claim is very outdated.
- Combination of WebAssembly + WebGPU is the enabling stack.
 - WebAssembly helps bring native libraries into the browser;
 - WebGPU brings GPU compute features that WebGL didn’t expose cleanly.
- Server-side vis still matters but has real deployment costs.